

PAPAYA Libraries

C++ and Python Reference

Both host APIs, every block, and the supported platforms.

V1.0.0



Engineered For Tech Digestion

www.shanon-technologies.com

Contents

1	About this reference	11
2	The programming model	12
2.1	A complete program	12
2.2	Five rules that are not obvious	13
2.3	Discretisation	13
2.4	Supported platforms	15
3	The PAPAYA class	17
3.1	Connecting	17
3.1.1	begin	17
3.1.2	end	18
3.1.3	setSPISettings	18
3.1.4	setTransportHooks	19
3.1.5	setCsDelayMicroseconds	19
3.1.6	Authenticate	19
3.2	Building a design	20
3.2.1	startDesign	20
3.2.2	stopDesign	21
3.2.3	startSampling	21
3.2.4	stopSampling	21
3.2.5	setMonitoringInterface	22
3.2.6	SetMonitorMode	22
3.2.7	SetLiveRate	23
3.3	Exchanging data	23
3.3.1	setInputs	23
3.3.2	getProbes	24
3.3.3	DescribeVectProbes	24
3.3.4	SetTraceSize	25
3.3.5	DownloadTrace	25
3.3.6	GetCaptureStatus	25
3.4	Status and identity	26
3.4.1	GetCurrentDesignStatus	26
3.4.2	GetCurrentComStatus	27
3.4.3	getSequenceNumber	27

3.4.4	getBoardInfo	27
3.4.5	GetBoardInfo	28
3.4.6	getDesignInstrumentation	28
3.4.7	describeDesign	29
3.4.8	remoteInputIndex	29
3.5	Stored designs	29
3.5.1	enableStorage	30
3.5.2	commitToEeprom	30
3.5.3	abortStorage	30
3.5.4	deleteApp	31
3.5.5	dumpApp	31
3.5.6	eraseAll	32
3.5.7	listApps	32
3.5.8	setBootApp	32
3.6	Datasets and audio	33
3.6.1	downloadDataset	33
3.6.2	downloadDatasetAll	34
3.6.3	downloadAudioSlot	34
3.6.4	downloadAudioSlotAll	35
3.6.5	eraseAllAudioSlots	35
3.6.6	uploadDataset	35
3.6.7	uploadAudioSlot	36
3.7	Errors and diagnostics	37
3.7.1	setComErrorHandler	37
3.7.2	setDesignErrorHandler	37
3.7.3	GetDebugLog	38
3.7.4	ClearDebugLog	38
3.7.5	enterBootloader	39
3.8	Relaying a board for someone else	39
3.8.1	The five transaction shapes	40
3.8.2	Sizes are yours to supply	40
3.8.3	relayCommand	40
3.8.4	relayCommandWithData	41
3.8.5	relayCommandNoResponse	42
3.8.6	relayBeginCommand	42
3.8.7	relaySendBodyChunk	43
3.8.8	relayAwaitResponse	43
3.8.9	relayReadDataPayload	44
3.8.10	relayReadProbes	44
3.8.11	relayReadVectorProbes	44
3.8.12	relayWriteInputs	45
3.8.13	resetBoard	45
3.9	Other services	46
4	Constructing blocks	48
4.1	Transfer functions (82)	48

4.1.1	CustFir	48
4.1.2	UpDwnperiod	48
4.1.3	absMax	49
4.1.4	absMin	49
4.1.5	absVal	49
4.1.6	adsr	49
4.1.7	allpass	49
4.1.8	amOscillator	50
4.1.9	autocorrLags	50
4.1.10	bitcrusher	50
4.1.11	butter	50
4.1.12	cheby1	50
4.1.13	cheby2	51
4.1.14	chorus	51
4.1.15	combFeedBack	51
4.1.16	combFeedForward	51
4.1.17	compressor	51
4.1.18	cosineFunc	52
4.1.19	cpuLoad	52
4.1.20	crestFactor	52
4.1.21	tf	52
4.1.22	node	53
4.1.23	macro	53
4.1.24	delayop	53
4.1.25	derivativeBlock	54
4.1.26	distortion	54
4.1.27	dwt	54
4.1.28	dwt_denoise	54
4.1.29	dwt_subband_energy	54
4.1.30	dwt_subband_gain	55
4.1.31	ecgBlRemove	55
4.1.32	ellip	55
4.1.33	emgEnvelope	55
4.1.34	entropy	55
4.1.35	envFollower	56
4.1.36	expFunc	56
4.1.37	fft	56
4.1.38	fftMagResolver	56
4.1.39	fft_HannWindow	57
4.1.40	fft_phaseResolver	57
4.1.41	fir	57
4.1.42	flanger	57
4.1.43	fracDelay	58
4.1.44	frictionStribeck	58
4.1.45	idwt	58
4.1.46	inputShaper	58

4.1.47	integratorBlock	58
4.1.48	interpolation	59
4.1.49	inverseBlock	59
4.1.50	leadLag	59
4.1.51	limiter	59
4.1.52	logFunc	59
4.1.53	maxObserver	60
4.1.54	maximum	60
4.1.55	mean	60
4.1.56	minObserver	60
4.1.57	minimum	60
4.1.58	momentaryPower	60
4.1.59	movingAverage	61
4.1.60	negateBlock	61
4.1.61	noise	61
4.1.62	noiseGate	61
4.1.63	notch	61
4.1.64	peak	62
4.1.65	peakToPeak	62
4.1.66	percentile	62
4.1.67	period	62
4.1.68	phaser	62
4.1.69	pid	63
4.1.70	pitchDetect	63
4.1.71	power	63
4.1.72	reverb	63
4.1.73	rootMeanSquare	63
4.1.74	signalSource	64
4.1.75	sineFunc	64
4.1.76	sqrMath	64
4.1.77	sqrtBlock	64
4.1.78	standardDeviation	64
4.1.79	stftFramer	65
4.1.80	tfd	65
4.1.81	variance	65
4.1.82	vuMeter	65
4.1.83	washout	65
4.1.84	windowFramer	65
4.1.85	zcr	66
4.2	Nodes (114)	66
4.2.1	RLS	66
4.2.2	RRLS	66
4.2.3	accelerometer	66
4.2.4	acimEKF	66
4.2.5	adaptiveKF	67
4.2.6	adaptiveNotch	67

4.2.7	adrc	67
4.2.8	ahrsEKF	67
4.2.9	analogIn	67
4.2.10	analogOut	68
4.2.11	atan2Block	68
4.2.12	backlashDeadBand	68
4.2.13	baroAltEKF	68
4.2.14	batterySOC_EKF	68
4.2.15	batterySOH_EKF	69
4.2.16	cepstrum	69
4.2.17	clarkeTransform	69
4.2.18	comp	69
4.2.19	compFilter	69
4.2.20	complementaryAhrs	70
4.2.21	deadZone	70
4.2.22	decibelsConverter	70
4.2.23	decisionTree	70
4.2.24	demux	70
4.2.25	dob	70
4.2.26	eegBandpower	71
4.2.27	emgHudgins	71
4.2.28	encoder	71
4.2.29	esc	71
4.2.30	escPID	71
4.2.31	expander	72
4.2.32	extSensor	72
4.2.33	featureStack	72
4.2.34	fuzzyPID	72
4.2.35	gain	72
4.2.36	gainSchedulePID	73
4.2.37	gainScheduler	73
4.2.38	gpsinEKF	73
4.2.39	gridSyncEKF	73
4.2.40	gyroscope	73
4.2.41	hjorth	73
4.2.42	hrvTime	74
4.2.43	ilc	74
4.2.44	imu	74
4.2.45	imuBiasEst	74
4.2.46	innovationChi2	74
4.2.47	integratorICBlock	75
4.2.48	invClarkeTransform	75
4.2.49	invParkTransform	75
4.2.50	linearKF	75
4.2.51	live2PidAw	75
4.2.52	livePiAw	75

4.2.53	livePidAw	76
4.2.54	lms	76
4.2.55	loadTorqueEKF	76
4.2.56	loudness	76
4.2.57	lqrServo	76
4.2.58	madgwickAhrs	76
4.2.59	mag2	77
4.2.60	mag3	77
4.2.61	magnetometer	77
4.2.62	mahonyAhrs	77
4.2.63	merger	77
4.2.64	microphone	77
4.2.65	mixerMultirotor	78
4.2.66	modulo	78
4.2.67	motor	78
4.2.68	mrac	78
4.2.69	mracLyapunov	78
4.2.70	mracPID	79
4.2.71	mux	79
4.2.72	parkTransform	79
4.2.73	pcaProject	79
4.2.74	perfMonitorPID	79
4.2.75	pidAw	80
4.2.76	pidFromModel	80
4.2.77	pmsmEKF	80
4.2.78	pwmIn	80
4.2.79	pwmOut	80
4.2.80	quantizer	81
4.2.81	quatFromEuler	81
4.2.82	quatMult	81
4.2.83	quatNormConj	81
4.2.84	quatToEuler	81
4.2.85	quatToRotmat	82
4.2.86	rateLimiter	82
4.2.87	recorder	82
4.2.88	relayAutoTune	82
4.2.89	repetitiveCtrl	82
4.2.90	replay	83
4.2.91	ringMod	83
4.2.92	robotLocEKF	83
4.2.93	sampleHold	83
4.2.94	sat	83
4.2.95	schmidt	83
4.2.96	sign	84
4.2.97	sinTrackEKF	84
4.2.98	skewKurt	84

4.2.99	smo	84
4.2.100	speaker	84
4.2.101	spectralFeatures	85
4.2.102	stateSpace	85
4.2.103	stepAutoTune	85
4.2.104	stereoMixer	85
4.2.105	stereoPan	85
4.2.106	strPID	85
4.2.107	str_	86
4.2.108	switchBlock	86
4.2.109	thermalEKF	86
4.2.110	tiltEKF	86
4.2.111	trajScurve	86
4.2.112	trajTrapezoid	87
4.2.113	wmrEKF	87
4.2.114	zScoreNorm	87
4.3	Macros (15)	87
4.3.1	ParamEQBand	87
4.3.2	ParamEQHigh	87
4.3.3	ParamEQPeak	88
4.3.4	butterBq	88
4.3.5	cheby1Bq	88
4.3.6	cheby2Bq	88
4.3.7	decimation	88
4.3.8	ellipBq	89
4.3.9	live2Pid	89
4.3.10	livePd	89
4.3.11	livePi	89
4.3.12	livePid	89
4.3.13	paramEQLow	90
4.3.14	rls	90
4.3.15	sineAm	90
5	The objects a program holds	91
5.1	probe	91
5.1.1	probe (constructor)	91
5.1.2	probe.getVal	92
5.1.3	probe.getId	92
5.1.4	probe.show	92
5.2	input	93
5.2.1	input (constructor)	93
5.2.2	input.enableRemote	93
5.2.3	input.addToQueue	94
5.2.4	input.getId	94
5.3	VectorProbe	95
5.3.1	VectorProbe (constructor)	95

5.3.2	VectorProbe.getSamples	95
5.3.3	VectorProbe.addToQueue	96
5.3.4	VectorProbe.show	96
5.3.5	VectorProbe.getId	96
6	Sources and outputs	97
6.1	The arguments every factory accepts	97
6.2	signalSource	97
6.3	analogOut	99
7	Opening the link from a host program	101
7.0.1	connect_usb	101
7.0.2	disconnect_usb	101
7.0.3	attach	102
8	Types	103
8.1	Enumerations	103
8.1.1	PP_AnOutMode	103
8.1.2	PP_C2dAlgo	103
8.1.3	PP_Cheby1Cfg	104
8.1.4	PP_Cheby2Cfg	104
8.1.5	PP_ComStatus	105
8.1.6	PP_DesignStatus	105
8.1.7	PP_EllipCfg	106
8.1.8	PP_EncoderOutput	106
8.1.9	PP_FilterType	106
8.1.10	PP_FirWindowType	107
8.1.11	PP_HwId	107
8.1.12	PP_MonInterface	109
8.1.13	PP_MonitorMode	109
8.1.14	PP_MotorOutput	110
8.1.15	PP_NodeType	110
8.1.16	PP_NoiseType	110
8.1.17	PP_SensorPartNumber	111
8.1.18	PP_ThresholdType	112
8.1.19	PP_Tune	113
8.1.20	PP_WaveForm	113
8.1.21	PP_WaveMode	113
8.1.22	PP_WaveMultisinePhase	114
8.1.23	PP_WaveShape	114
8.1.24	PP_WaveTrigPol	115
8.1.25	PP_WaveletType	115
8.2	Configuration structures	116
8.2.1	PP_ACIM_EKF	116
8.2.2	PP_ADRCCConfig	117
8.2.3	PP_AHRS_EKF	117
8.2.4	PP_ActuatorSpec	117

8.2.5	PP_AdaptiveNotchConfig	117
8.2.6	PP_BaroAlt_EKF	118
8.2.7	PP_BatterySOC_EKF	118
8.2.8	PP_BatterySOH_EKF	119
8.2.9	PP_DOBConfig	119
8.2.10	PP_ESCConfig	119
8.2.11	PP_ESC_PID_Config	120
8.2.12	PP_FuzzyPIDConfig	120
8.2.13	PP_GPSINS_EKF	121
8.2.14	PP_GainSchedulePIDConfig	121
8.2.15	PP_GainSchedulerConfig	121
8.2.16	PP_GridSync_EKF	122
8.2.17	PP_ILCConfig	122
8.2.18	PP_LMSCConfig	122
8.2.19	PP_LoadTorque_EKF	122
8.2.20	PP_MRACConfig	123
8.2.21	PP_MRACLyapunovConfig	123
8.2.22	PP_MRAC_PID_Config	123
8.2.23	PP_PMSM_EKF	124
8.2.24	PP_PerfMonPID_Config	124
8.2.25	PP_RLSPParam	125
8.2.26	PP_RRLSPParam	125
8.2.27	PP_RelayAutoTuneConfig	126
8.2.28	PP_RepCtrlConfig	126
8.2.29	PP_RobotLoc_EKF	126
8.2.30	PP_SMO	127
8.2.31	PP_STRConfig	127
8.2.32	PP_STR_PID_Config	127
8.2.33	PP_SensorSpec	128
8.2.34	PP_SinTrack_EKF	128
8.2.35	PP_StepAutoTuneConfig	128
8.2.36	PP_Thermal_EKF	129
8.2.37	PP_Tilt_EKF	129
8.2.38	PP_WMR_EKF	129
8.2.39	PP_adaptiveKF	130
8.2.40	PP_anInCfg	130
8.2.41	PP_anOutCfg	131
8.2.42	PP_compFilter	131
8.2.43	PP_encoderBlockCfg	131
8.2.44	PP_encoderCfg	131
8.2.45	PP_linearKF	132
8.2.46	PP_motorBlockCfg	132
8.2.47	PP_motorCfg	133
8.2.48	PP_pwmInCfg	133
8.2.49	PP_pwmOutCfg	133
8.2.50	PP_stateSpace	134

9 Linking blocks	135
10 Configuration structures	136

Chapter 1

About this reference

This volume documents the two host libraries that drive a PAPAYA VL board: the **Python package** `papaya` and the **C++ library** `papaya_lib`. It is an API reference – what you can call, what you pass it, what it returns, and what a working program looks like.

Who it is for. Developers writing host software. If you are drawing diagrams in the Studio rather than writing code, you want *PAPAYA Studio – User Manual* instead.

What it does not cover. What each block *computes* – its mathematics, ports and validity envelope – is in *PAPAYA Studio – User Manual*. The wire protocol beneath these libraries is in *DLP Protocol – Programmer Manual*. Electrical limits are in *PAPAYA VL – Datasheet*.

Chapter 2

The programming model

The diagram runs on the board. Your program runs wherever you like.

A host program does three things: it *describes* a block diagram to the board, it *starts* the board sampling, and then it *exchanges values* with the running diagram – writing inputs, reading probes.

The two libraries expose the same shape, so the reference covers both together and shows each call in Python and C++.

2.1 A complete program

```
from papaya import *
from papaya.usb_transport import connect_usb

if not connect_usb():           # opens the link. Do this FIRST
    raise SystemExit("no PAPAYA board answered over USB")

board = PAPAYA(0)               # board id 0

board.startDesign(1000.0, PP_C2dAlgo.Tustin) # Fs and discretisation FIRST

src = signalSource.sine(freq=22)
filt = butter(5, 10, PP_FilterType.passLow)
out = probe("out")

src > filt                      # link: output -> input
filt > out

board.stopDesign()              # end of description
board.startSampling()           # the diagram now runs

for _ in range(100):
    board.readProbesLive()      # take one pushed batch
    print(out.getVal())
```

```
#include "PAPAYA.h"

PAPAYA board(0);

void setup() {
    board.begin(0, SPI, PP_CS_PIN, SPISettings(8000000, MSBFIRST, SPI_MODE0));
    board.startDesign(1000.0f, Tustin);

    signalSource src = signalSource::sine(22.0f);
    butter      filt(5, 10.0f, passLow);
    probe      out("out");

    src > filt;
    filt > out;

    board.stopDesign();
    board.startSampling();
}

void loop() {
    board.getProbes();
    float v = out.getVal();
}
```

2.2 Five rules that are not obvious

1. **startDesign must precede every block.** Continuous blocks are discretised *as they are constructed*, from the sampling frequency and algorithm startDesign set. Construct a block before it, or change the rate after, and the coefficients are silently different.
2. **An output drives exactly one input.** A second link from the same output is refused. Fan out with expander, which is why it appears in 73 of the 82 shipped examples.
3. **A block enters the execution order when its last output is linked**, not when it is constructed. This is why a hardware output actuates one sampling period behind the block feeding it.
4. **Enum members are qualified in Python.** PP_FilterType.passLow, never a bare passLow; the package exports the enum type, not its members. In C++ the enumerators are unqualified.
5. **butter, tf and the rest are classes.** Link *instances*. Writing src > butter compares against the class object: a compile error in C++, a TypeError in Python.

2.3 Discretisation

startDesign takes exactly three algorithms: **Tustin**, **Euler Backward** and **Euler Forward**. The board performs the discretisation.

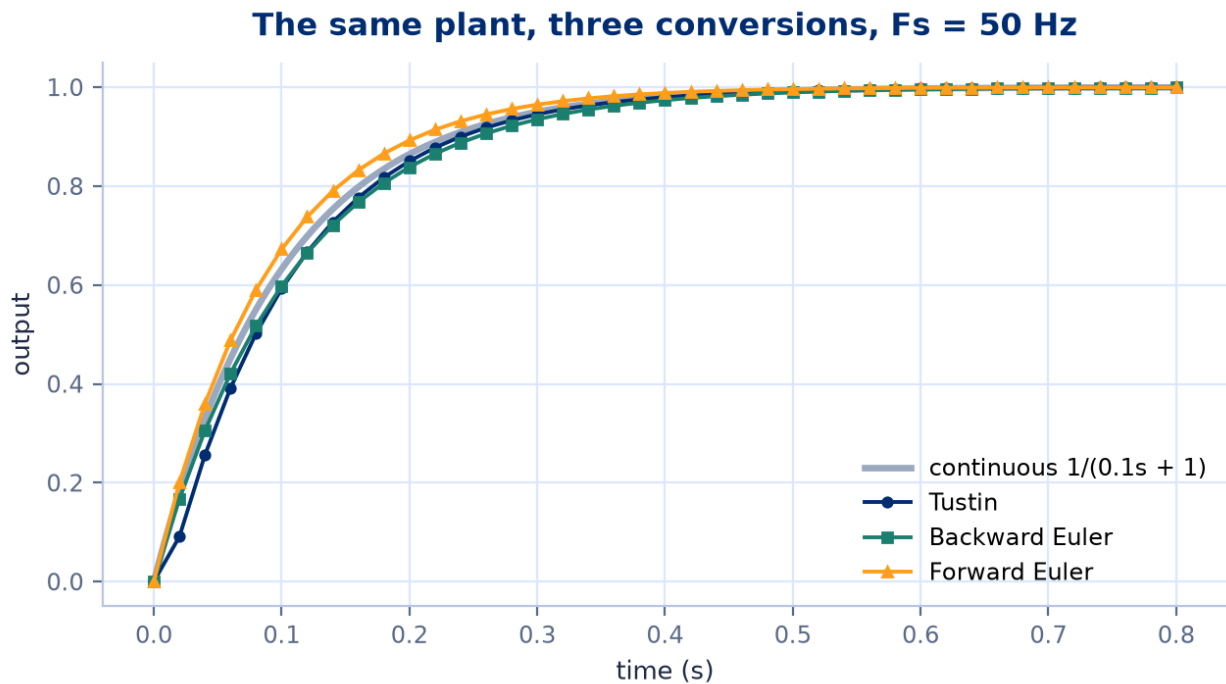


Figure 2.1: The same continuous plant converted by each of the three methods the product offers, at 50 Hz, against the exact continuous response. The curves are the difference equations themselves, simulated.

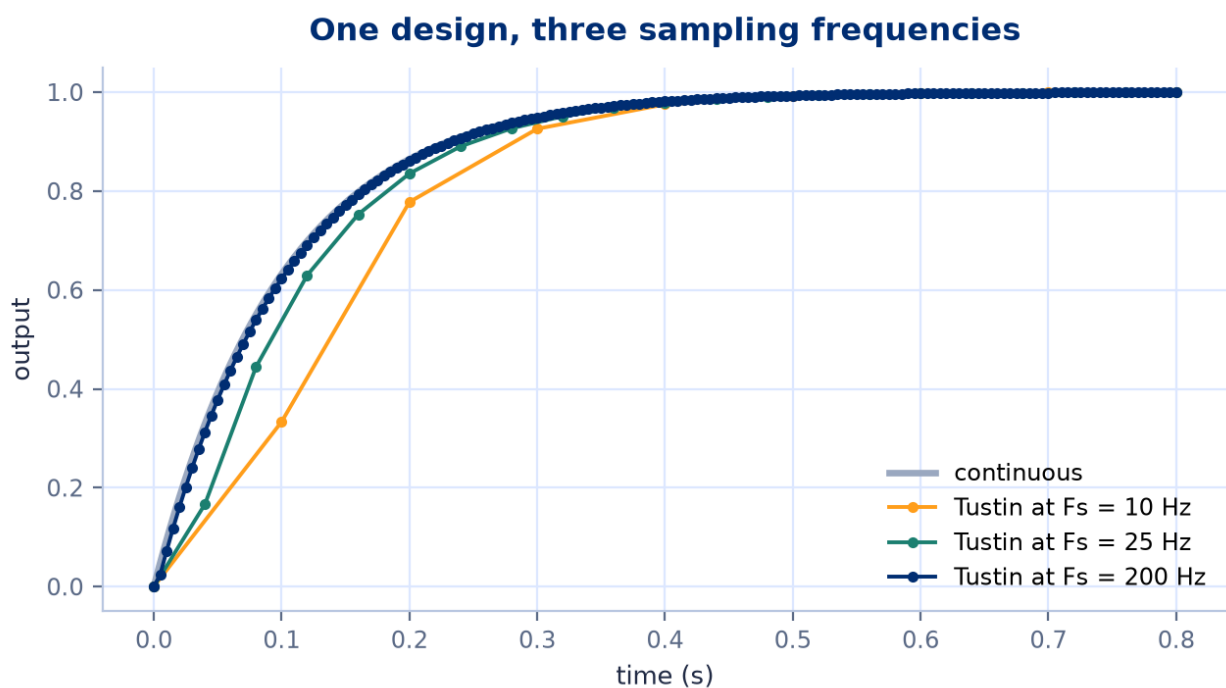


Figure 2.2: One design at three sampling frequencies. The sampling frequency is part of the design, not a detail of the implementation: the same continuous model produces measurably different behaviour at each rate.

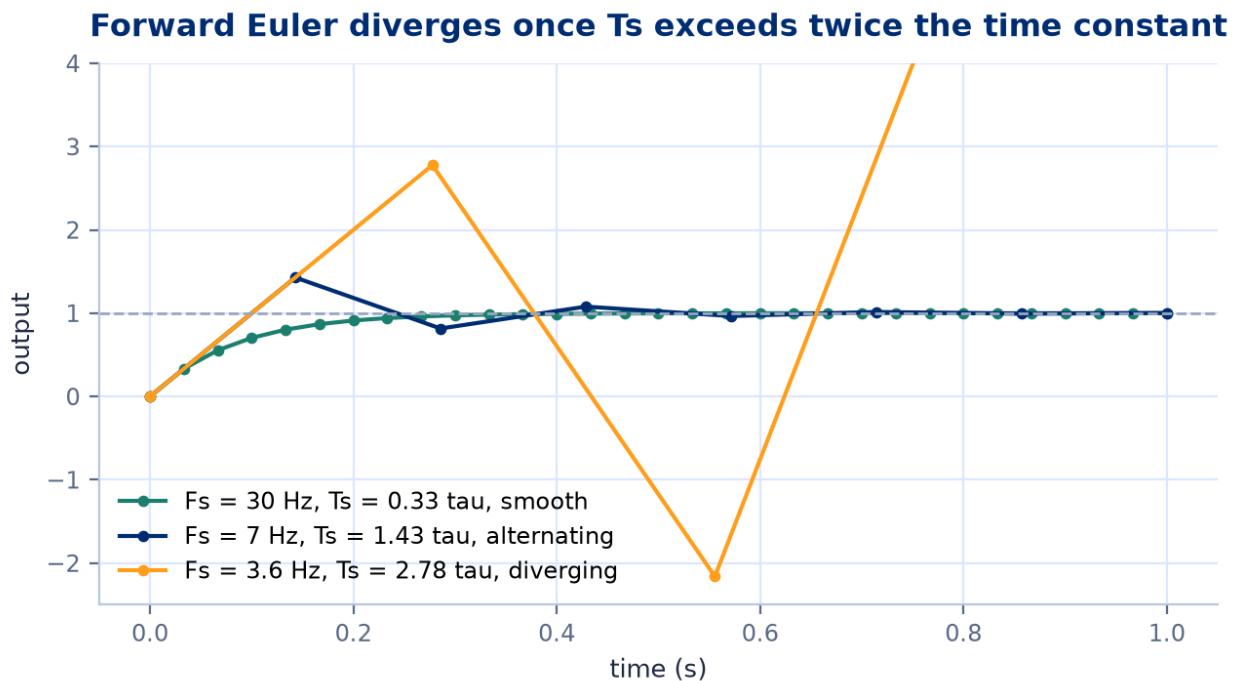


Figure 2.3: Forward Euler is conditionally stable. Its pole is $1 - T_s/\tau$, so once the sample period exceeds twice the time constant the pole leaves the unit circle and the output grows without bound, while the continuous design it came from is unchanged and well behaved.

Warning

The System Identification tooling uses **ZOH** and **bilinear** internally. Neither is available here, and an unrecognised value **silently falls back to Tustin** rather than raising.

2.4 Supported platforms

Target	Transport port
PC (Windows / Linux)	pc_usb, windows_serial, posix_serial
Raspberry Pi	linux_spidev
Arduino	arduino_spi, host_arduino_compat
STM32Cube	stm32cubeide_hal
USB bridge	usb_bridge

Up to **4 boards** share one SPI master, addressed 0-3.

Note

Those are hosts that *drive* a PAPAYA VL. The real-time diagram always executes on the board.

Chapter 3

The PAPAYA class

One object per board; every service below is a method on it. Each service is specified in the same form - its name, its syntax in both languages, its parameters by direction, what it returns, what it does, and where it is declared.

3.1 Connecting

Opening and closing the link to a board.

3.1.1 begin

[PPL_API_105]:

Service Name	begin
Syntax (Python)	<code>begin(Id: 'int') -> 'None'</code>
Syntax (C++)	<code>bool begin(int Id)</code> <code>bool begin(uint8_t boardId,</code> <code>PP_TransportTxBytesFn txFunction,</code> <code>PP_TransportRxBytesFn rxFunction)</code> <code>bool begin(int Id, SPIClass &spi, uint8_t</code> <code>csPin, const SPISettings &settings, bool</code> <code>callSpiBegin = true, uint16_t csDelayUs =</code> <code>PP_DEFAULT_CS_DELAY_US)</code>
Parameters (in)	- Id (int, 0 to 3)
Parameters (inout)	None
Parameters (out)	- spi
Return value	Processing status TRUE: the board answered and the link is open. FALSE: no board answered, and no further call will succeed.

Service Name	begin
Description	Open the link to a board. Call once before anything else.
Available via	papaya (Python), PAPAYA.h (C++)

3.1.2 end

[PPL_API_110]:

Service Name	end
Syntax (Python)	end() -> 'None'
Syntax (C++)	void end()
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Close the link and release the transport.
Available via	papaya (Python), PAPAYA.h (C++)

3.1.3 setSPISettings

[PPL_API_115]:

Service Name	setSPISettings
Syntax (C++)	void setSPISettings(const SPISettings &settings)
Parameters (in)	- &settings (const SPISettings)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Set the clock rate, bit order and mode used on the SPI link. Call it before begin; changing it afterwards does not move an open link.
Available via	PAPAYA.h (C++)

3.1.4 setTransportHooks

[PPL_API_120]:

Service Name	setTransportHooks
Syntax (C++)	<code>void setTransportHooks(const PP_TransportHooks &hooks)</code>
Parameters (in)	- &hooks (const PP_TransportHooks)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Supply your own send and receive functions, so a board can be reached over a link the library does not drive itself.
Available via	PAPAYA.h (C++)

3.1.5 setCsDelayMicroseconds

[PPL_API_125]:

Service Name	setCsDelayMicroseconds
Syntax (C++)	<code>void setCsDelayMicroseconds(uint16_t csDelayUs)</code>
Parameters (in)	- csDelayUs (uint16_t, microseconds, 0 to 65535, default 200)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Hold time in microseconds between asserting chip select and the first clock edge. Raise it when a long cable or a level shifter corrupts the first byte of every frame.
Available via	PAPAYA.h (C++)

3.1.6 Authenticate

[PPL_API_130]:

Service Name	Authenticate
Syntax (Python)	<code>Authenticate(nonce: 'Optional[bytes]' = None) -> 'bytes'</code>

Service Name	Authenticate
Parameters (in)	- nonce (Optional[bytes], default None)
Parameters (inout)	None
Parameters (out)	None
Return value	bytes - the response the board expects.
Description	Perform the challenge-response the board requires before it will operate.
Available via	papaya (Python)

3.2 Building a design

Deploying a block diagram and running it.

3.2.1 startDesign

[PPL_API_205]:

Service Name	startDesign
Syntax (Python)	startDesign(SamplingFreq: 'float', Algo: 'PP_C2dAlgo') -> 'None'
Syntax (C++)	void startDesign(float SamplingFreq, PP_C2dAlgo Algo)
Parameters (in)	- SamplingFreq (float, Hz, 0.1 to 50 000 ; outside that the board refuses the design with SamplingFreqOutOfLimit) - Algo (PP_C2dAlgo, see the PP_C2dAlgo table)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Set the sampling frequency and discretisation algorithm, and open a design. Must precede every block construction.
Available via	papaya (Python), PAPAYA.h (C++)

3.2.2 stopDesign

[PPL_API_210]:

Service Name	stopDesign
Syntax (Python)	stopDesign() -> 'None'
Syntax (C++)	void stopDesign(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Close the description. No further blocks or links are accepted.
Available via	papaya (Python), PAPAYA.h (C++)

3.2.3 startSampling

[PPL_API_215]:

Service Name	startSampling
Syntax (Python)	startSampling() -> 'None'
Syntax (C++)	void startSampling(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Begin executing the deployed diagram in real time.
Available via	papaya (Python), PAPAYA.h (C++)

3.2.4 stopSampling

[PPL_API_220]:

Service Name	stopSampling
Syntax (Python)	stopSampling() -> 'None'
Syntax (C++)	void stopSampling(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None

Service Name	stopSampling
Return value	None
Description	Halt execution. The design stays on the board.
Available via	papaya (Python), PAPAYA.h (C++)

3.2.5 setMonitoringInterface

[PPL_API_225]:

Service Name	setMonitoringInterface
Syntax (Python)	setMonitoringInterface(interf: 'PP_MonInterface') -> 'None'
Syntax (C++)	void setMonitoringInterface(PP_MonInterface InterfaceId)
Parameters (in)	- interf (PP_MonInterface)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Choose which physical link carries monitoring traffic.
Available via	papaya (Python), PAPAYA.h (C++)

3.2.6 SetMonitorMode

[PPL_API_230]:

Service Name	SetMonitorMode
Syntax (Python)	SetMonitorMode(mode: 'PP_MonitorMode') -> 'None'
Syntax (C++)	bool setMonitorMode(PP_MonitorMode mode)
Parameters (in)	- mode (PP_MonitorMode)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Live streaming, offline capture, or off.
Available via	papaya (Python), PAPAYA.h (C++, spelled setMonitorMode)

3.2.7 SetLiveRate

[PPL_API_235]:

Service Name	SetLiveRate
Syntax (Python)	SetLiveRate(hz: 'int') -> 'None'
Parameters (in)	- hz (int)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Deprecated, and it cannot succeed: the board refuses this call in every case, because the samples-per-probe figure it used to set is fixed in firmware at 128 per probe. Worse, the refusal latches, so the board reports the failure on every later response until a startDesign clears it, and the call therefore breaks the next command as well as failing itself. Remove the call; there is nothing to replace it with because there is nothing left to set. It is not in the C++ library.
Available via	papaya (Python)

3.3 Exchanging data

Writing inputs and reading probes while sampling.

3.3.1 setInputs

[PPL_API_305]:

Service Name	setInputs
Syntax (C++)	void setInputs(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Push every input block's current value to the board.
Available via	PAPAYA.h (C++)

3.3.2 getProbes

[PPL_API_310]:

Service Name	getProbes
Syntax (C++)	void getProbes(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Refresh every probe from the board. Call before reading values.
Available via	PAPAYA.h (C++)

3.3.3 DescribeVectProbes

[PPL_API_315]:

Service Name	DescribeVectProbes
Syntax (Python)	DescribeVectProbes() -> 'None'
Syntax (C++)	bool describeVectProbes(PP_VectorProbeDesc *out, uint8_t capacity, uint8_t *count = nullptr)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	- out - count
Return value	None
Description	Ask the board to describe its vector probes: how many there are, and the length and kind of each. Read it once after deploying a design, so a reader knows the shape of the frames that follow.
Available via	papaya (Python), PAPAYA.h (C++, spelled describeVectProbes)

3.3.4 SetTraceSize

[PPL_API_320]:

Service Name	SetTraceSize
Syntax (Python)	SetTraceSize(total_samples: 'int') -> 'None'
Syntax (C++)	bool setTraceSize(uint32_t totalSamples)
Parameters (in)	- total_samples (int, samples)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Sample budget for an offline capture.
Available via	papaya (Python), PAPAYA.h (C++, spelled setTraceSize)

3.3.5 DownloadTrace

[PPL_API_325]:

Service Name	DownloadTrace
Syntax (Python)	DownloadTrace(offset: 'int', length: 'int') -> 'None'
Syntax (C++)	bool downloadTrace(uint32_t offset, uint16_t length, uint8_t *buffer)
Parameters (in)	- offset (int, bytes) - length (int, bytes)
Parameters (inout)	None
Parameters (out)	- buffer
Return value	None
Description	Fetch part of a completed offline capture.
Available via	papaya (Python), PAPAYA.h (C++, spelled downloadTrace)

3.3.6 GetCaptureStatus

[PPL_API_330]:

Service Name	GetCaptureStatus
Syntax (Python)	GetCaptureStatus() -> 'None'

Service Name	GetCaptureStatus
Syntax (C++)	<code>bool getCaptureStatus(PP_CaptureStatus &status)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	- status
Return value	None
Description	Ask how far an offline capture has progressed: how many samples are already in memory, and whether the capture has finished. A trace may only be downloaded once it has.
Available via	papaya (Python), PAPAYA.h (C++, spelled <code>getCaptureStatus</code>)

3.4 Status and identity

What the board reports about itself.

3.4.1 GetcurrentDesignStatus

[PPL_API_405]:

Service Name	GetcurrentDesignStatus
Syntax (Python)	<code>GetcurrentDesignStatus() -> 'PP_DesignStatus'</code>
Syntax (C++)	<code>PP_DesignStatus GetcurrentDesignStatus(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	PP_DesignStatus. A value of 8 means the board rejected the design.
Description	The board's view of the deployed design. Status 8 means the design was rejected.
Available via	papaya (Python), PAPAYA.h (C++)

3.4.2 GetcurrentComStatus

[PPL_API_410]:

Service Name	GetcurrentComStatus
Syntax (Python)	GetcurrentComStatus() -> 'PP_ComStatus'
Syntax (C++)	PP_ComStatus GetcurrentComStatus(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	PP_ComStatus - the state of the link.
Description	The state of the communication link.
Available via	papaya (Python), PAPAYA.h (C++)

3.4.3 getSequenceNumber

[PPL_API_415]:

Service Name	getSequenceNumber
Syntax (Python)	getSequenceNumber() -> 'int'
Syntax (C++)	uint16_t getSequenceNumber(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	uint16_t - the command counter.
Description	The command counter, for diagnosing a dropped exchange.
Available via	papaya (Python), PAPAYA.h (C++)

3.4.4 getBoardInfo

[PPL_API_420]:

Service Name	getBoardInfo
Syntax (C++)	bool getBoardInfo(PP_BoardInfo &info)
Parameters (in)	- &info (PP_BoardInfo)
Parameters (inout)	None
Parameters (out)	- info
Return value	Processing status TRUE: info is filled and may be used. FALSE: info must be ignored.

Service Name	getBoardInfo
Description	Board identity, part number and firmware version.
Available via	PAPAYA.h (C++)

3.4.5 GetBoardInfo

[PPL_API_425]:

Service Name	GetBoardInfo
Syntax (Python)	GetBoardInfo() -> 'None'
Syntax (C++)	bool getBoardInfo(PP_BoardInfo &info)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	- info
Return value	None
Description	Board identity, part number and firmware version.
Available via	papaya (Python), PAPAYA.h (C++, spelled getBoardInfo)

3.4.6 getDesignInstrumentation

[PPL_API_430]:

Service Name	getDesignInstrumentation
Syntax (Python)	getDesignInstrumentation(verbose: 'bool' = True) -> 'Optional[dict]'
Syntax (C++)	bool getDesignInstrumentation(PP_DesignInstrumentation &info)
Parameters (in)	- verbose (bool, default True)
Parameters (inout)	None
Parameters (out)	- info
Return value	Processing status TRUE: info is filled and may be used. FALSE: info must be ignored.
Description	Per-block timing and load for the running design.
Available via	papaya (Python), PAPAYA.h (C++)

3.4.7 describeDesign

[PPL_API_435]:

Service Name	describeDesign
Syntax (C++)	<code>bool describeDesign(PP_RemoteDesign &design)</code>
Parameters (in)	- &design (PP_RemoteDesign)
Parameters (inout)	None
Parameters (out)	- design
Return value	Processing status TRUE: design describes what the board currently holds. FALSE: design must be ignored.
Description	Read back the design the board currently holds.
Available via	PAPAYA.h (C++)

3.4.8 remoteInputIndex

[PPL_API_440]:

Service Name	remoteInputIndex
Syntax (C++)	<code>static int remoteInputIndex(const PP_RemoteDesign &design, const char *name)</code>
Parameters (in)	- &design (const PP_RemoteDesign) - *name (const char)
Parameters (inout)	None
Parameters (out)	None
Return value	int - the index, or a negative value if the name is not present in the design.
Description	Index of a named remote input within a described design.
Available via	PAPAYA.h (C++)

3.5 Stored designs

Designs kept in EEPROM and replayed with no host.

3.5.1 enableStorage

[PPL_API_505]:

Service Name	enableStorage
Syntax (Python)	<code>enableStorage(label: 'str') -> 'None'</code>
Syntax (C++)	<code>void enableStorage(const char* label)</code>
Parameters (in)	- label (str)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Begin capturing the next design into EEPROM under a label.
Available via	papaya (Python), PAPAYA.h (C++)

3.5.2 commitToEeprom

[PPL_API_510]:

Service Name	commitToEeprom
Syntax (Python)	<code>commitToEeprom() -> 'None'</code>
Syntax (C++)	<code>void commitToEeprom(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Write the captured design to EEPROM.
Available via	papaya (Python), PAPAYA.h (C++)

3.5.3 abortStorage

[PPL_API_515]:

Service Name	abortStorage
Syntax (Python)	<code>abortStorage() -> 'None'</code>
Syntax (C++)	<code>void abortStorage(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None

Service Name	abortStorage
Return value	None
Description	Discard a capture in progress.
Available via	papaya (Python), PAPAYA.h (C++)

3.5.4 deleteApp

[PPL_API_520]:

Service Name	deleteApp
Syntax (Python)	<code>deleteApp(app_id: 'int') -> 'None'</code>
Syntax (C++)	<code>void deleteApp(uint16_t appId)</code>
Parameters (in)	- app_id (int, one of up to 16 stored applications)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Remove a stored design by id.
Available via	papaya (Python), PAPAYA.h (C++)

3.5.5 dumpApp

[PPL_API_525]:

Service Name	dumpApp
Syntax (Python)	<code>dumpApp(app_id: 'int') -> 'None'</code>
Syntax (C++)	<code>void dumpApp(uint16_t appId)</code>
Parameters (in)	- app_id (int, one of up to 16 stored applications)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Return the contents of one stored application. Available over USB only; an SPI master asking for it is refused.
Available via	papaya (Python), PAPAYA.h (C++)

3.5.6 eraseAll

[PPL_API_530]:

Service Name	eraseAll
Syntax (Python)	<code>eraseAll()</code> -> 'None'
Syntax (C++)	<code>void eraseAll(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Erase every stored design.
Available via	<code>papaya</code> (Python), <code>PAPAYA.h</code> (C++)

3.5.7 listApps

[PPL_API_535]:

Service Name	listApps
Syntax (Python)	<code>listApps()</code> -> 'None'
Syntax (C++)	<code>void listApps(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Ask the board for its directory of stored applications: which slots hold a design, and what each one is called.
Available via	<code>papaya</code> (Python), <code>PAPAYA.h</code> (C++)

3.5.8 setBootApplication

[PPL_API_540]:

Service Name	setBootApplication
Syntax (Python)	<code>setBootApplication(boot_id: 'int', auto_load: 'bool' = True)</code> -> 'None'
Syntax (C++)	<code>void setBootApplication(uint16_t bootId, bool autoLoad)</code>

Service Name	setBootTest
Parameters (in)	- boot_id (int, one of up to 16 stored applications) - auto_load (bool, default True)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Choose which stored application the board loads by itself at power-up, and whether it loads one at all.
Available via	papaya (Python), PAPAYA.h (C++)

3.6 Datasets and audio

Bulk data in the board's slots.

3.6.1 downloadDataset

[PPL_API_605]:

Service Name	downloadDataset
Syntax (Python)	downloadDataset(slot_id: 'int', offset: 'int', length: 'int') -> 'Optional[bytes]'
Syntax (C++)	bool downloadDataset(uint8_t slotId, uint32_t offset, uint16_t length, uint8_t *buffer)
Parameters (in)	- slot_id (int, 0 to 7) - offset (int, bytes, 4 per sample) - length (int, bytes, 4 per sample)
Parameters (inout)	None
Parameters (out)	- buffer
Return value	bytes on success, or None if the board did not answer.
Description	Read one chunk of a dataset slot. offset and length are in bytes , and the payload is 32-bit floats, so one sample is four bytes. Prefer downloadDatasetAll unless you are paging a large slot yourself.
Available via	papaya (Python), PAPAYA.h (C++)

3.6.2 downloadDatasetAll

[PPL_API_610]:

Service Name	downloadDatasetAll
Syntax (Python)	<code>downloadDatasetAll(slot_id: 'int', length_samples: 'int') -> 'Optional[bytes]'</code>
Parameters (in)	- slot_id (int, 0 to 7) - length_samples (int, samples)
Parameters (inout)	None
Parameters (out)	None
Return value	bytes on success, or None if the board did not answer.
Description	Read a whole dataset slot in one call, paging it for you. length_samples is a count of samples , not bytes.
Available via	papaya (Python)

3.6.3 downloadAudioSlot

[PPL_API_615]:

Service Name	downloadAudioSlot
Syntax (Python)	<code>downloadAudioSlot(slot_id: 'int', offset: 'int', length: 'int') -> 'Optional[bytes]'</code>
Syntax (C++)	<code>bool downloadAudioSlot(uint8_t slotId, uint32_t offset, uint16_t length, uint8_t *buffer)</code>
Parameters (in)	- slot_id (int, 0 to 7) - offset (int, bytes, 4 per sample) - length (int, bytes, 4 per sample)
Parameters (inout)	None
Parameters (out)	- buffer
Return value	bytes on success, or None if the board did not answer.
Description	Read one chunk of an audio slot. offset and length are in bytes , four bytes to a sample.
Available via	papaya (Python), PAPAYA.h (C++)

3.6.4 downloadAudioSlotAll

[PPL_API_620]:

Service Name	downloadAudioSlotAll
Syntax (Python)	downloadAudioSlotAll(slot_id: 'int', length_samples: 'int') -> 'Optional[bytes]'
Parameters (in)	- slot_id (int, 0 to 7) - length_samples (int, samples)
Parameters (inout)	None
Parameters (out)	None
Return value	bytes on success, or None if the board did not answer.
Description	Read a whole audio slot in one call. length_samples is a count of samples , not bytes.
Available via	papaya (Python)

3.6.5 eraseAllAudioSlots

[PPL_API_625]:

Service Name	eraseAllAudioSlots
Syntax (Python)	eraseAllAudioSlots() -> 'bool'
Syntax (C++)	bool eraseAllAudioSlots(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	Processing status TRUE: every slot was erased. - FALSE: the erase did not complete.
Description	Erase every audio slot on the board. Nothing is kept and there is no undo.
Available via	papaya (Python), PAPAYA.h (C++)

3.6.6 uploadDataset

[PPL_API_630]:

Service Name	uploadDataset
Syntax (Python)	<code>uploadDataset(slot_id: 'int', samples, *, sample_rate_hz: 'Optional[int]' = None, timestamp_unix: 'int' = 0, fmt: 'Optional[int]' = None) -> 'bool'</code>
Syntax (C++)	<code>bool uploadDataset(uint8_t slotId, const float *samples, uint32_t count, uint32_t sampleRateHz = 0u)</code>
Parameters (in)	- slot_id (int, 0 to 7) - samples - sample_rate_hz (Optional[int], default None, Hz) - timestamp_unix (int, default 0, seconds since the Unix epoch) - fmt (Optional[int], default None, 0 for float32)
Parameters (inout)	None
Parameters (out)	None
Return value	bool - TRUE on success, FALSE if the board did not accept the command.
Description	Write a recording into a dataset slot. Most programs never call this: a <code>signalSource.dataset(samples)</code> block uploads its own data when the design is deployed.
Available via	papaya (Python), PAPAYA.h (C++)

3.6.7 uploadAudioSlot

[PPL_API_635]:

Service Name	uploadAudioSlot
Syntax (Python)	<code>uploadAudioSlot(slot_id: 'int', samples: 'bytes', sample_rate_hz: 'int', timestamp_unix: 'int' = 0, fmt: 'Optional[int]' = None) -> 'bool'</code>
Parameters (in)	- slot_id (int, 0 to 7) - samples (bytes) - sample_rate_hz (int, Hz) - timestamp_unix (int, default 0, seconds since the Unix epoch) - fmt (Optional[int], default None, 0 for float32)
Parameters (inout)	None
Parameters (out)	None
Return value	bool - TRUE on success, FALSE if the board did not accept the command.

Service Name	uploadAudioSlot
Description	Write a recording into an audio slot. <code>samples</code> is the raw payload, 32-bit floats, little-endian, and <code>sample_rate_hz</code> is the rate it was recorded at rather than the rate it will be played at.
Available via	<code>papaya</code> (Python)

3.7 Errors and diagnostics

Handlers, logs and the bootloader.

3.7.1 setComErrorHandler

[PPL_API_705]:

Service Name	setComErrorHandler
Syntax (Python)	<code>setComErrorHandler(handler: 'Optional[Callable[[], None]]') -> 'None'</code>
Syntax (C++)	<code>void setComErrorHandler(PP_ErrorHandlerType UserHandler)</code>
Parameters (in)	- <code>handler</code> (Optional[Callable[[], None]])
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Install a callback for transport failures.
Available via	<code>papaya</code> (Python), <code>PAPAYA.h</code> (C++)

3.7.2 setDesignErrorHandler

[PPL_API_710]:

Service Name	setDesignErrorHandler
Syntax (Python)	<code>setDesignErrorHandler(handler: 'Optional[Callable[[], None]]') -> 'None'</code>
Syntax (C++)	<code>void setDesignErrorHandler(PP_ErrorHandlerType UserHandler)</code>
Parameters (in)	- <code>handler</code> (Optional[Callable[[], None]])

Service Name	setDesignErrorHandler
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Install a callback for design rejections.
Available via	papaya (Python), PAPAYA.h (C++)

3.7.3 GetDebugLog

[PPL_API_715]:

Service Name	GetDebugLog
Syntax (Python)	GetDebugLog() -> 'None'
Syntax (C++)	bool getDebugLog(char *buffer, uint32_t capacity)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	- buffer
Return value	None
Description	Take a snapshot of the board's debug log. Read it when a design has been refused and the status number alone does not say why.
Available via	papaya (Python), PAPAYA.h (C++, spelled getDebugLog)

3.7.4 ClearDebugLog

[PPL_API_720]:

Service Name	ClearDebugLog
Syntax (Python)	ClearDebugLog() -> 'None'
Syntax (C++)	bool clearDebugLog(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Empty the board's debug log, so the next snapshot holds only what happened after this call.

Service Name	ClearDebugLog
Available via	papaya (Python), PAPAYA.h (C++, spelled <code>clearDebugLog</code>)

3.7.5 enterBootloader

[PPL_API_725]:

Service Name	enterBootloader
Syntax (Python)	<code>enterBootloader()</code> -> 'None'
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Restart into the firmware bootloader.
Available via	papaya (Python)

3.8 Relaying a board for someone else

Every service above builds its own command: your code decides what to say and the library says it. Relaying is the opposite case. The bytes arrive already formed, over whatever link your host speaks, and its only job is to put them on the board's wire correctly and bring the answer back. It builds nothing and interprets nothing.

That is what these are for, and it is a narrow purpose: a host that carries PAPAYA Studio's conversation to a board it is wired to, so the board can be driven from a computer that is not attached to it. If you are writing an ordinary program, use the services above instead.

Timing is not yours here. The wait after chip-select, the size a body is chunked into, the interval and the deadline of the response poll: all of them belong to the board and stay inside the library. Nothing below exposes them, deliberately, because a second copy of those figures is a second thing to keep in step with the board.

The sequence number is the far end's. Every relayed command carries the number whoever composed it used, and the library takes it out of the header you hand over. This is worth stating because getting it wrong does not fail loudly: the board answers perfectly well, the reply fails its sequence check, and every command appears to time out on a link that is working.

A command must never be left half sent. The board has no timeout between the pieces of one command. A body that stops arriving leaves it waiting forever, and nothing short of a reset clears that. If your host cannot finish a body, send the rest as filler anyway and read the error the board then answers with. An error is recoverable; a board waiting forever is not.

Two commands are refused before they reach the wire. The flow-credit command reads from the USB endpoint in firmware and means nothing on the board's other wire. The deprecated live-rate command is rejected unconditionally and the rejection latches, so it breaks the next command as well as itself. A relay must not send either.

3.8.1 The five transaction shapes

There are only five ways a command is carried on this wire, and a relay needs no other knowledge of the protocol at all. That is the point: a relay that parsed commands would have to keep up with a firmware it does not model.

Shape	What it is	Use
1	Header, body, then poll for the status	<code>relayCommand</code> , or the three-step form below
2	Shape 1, then read the payload the board queued behind the status	<code>relayCommandWithData</code>
3	Read probe samples by explicit byte count	<code>relayReadProbes</code> , <code>relayReadVectorProbes</code>
4	Write input values, all in one exchange	<code>relayWriteInputs</code>
5	Header and body, and no poll at all	<code>relayCommandNoResponse</code>

Shape 5 exists because a command that reboots the board is gone before it could answer. Polling for a status spends the whole deadline and then reports a failure for a command that did exactly what it was asked.

3.8.2 Sizes are yours to supply

`getProbes()` sizes itself from the probe objects your program constructed. A relay constructs none, so it would ask the board for zero bytes and get nothing back. Nothing on this wire announces a length either. So every relay read takes an explicit byte count, and that count has to come from the end that knows the design.

3.8.3 `relayCommand`

[PPL_API_730]:

Service Name	<code>relayCommand</code>
Syntax (C++)	<code>bool relayCommand(const uint8_t *header, const uint8_t *body, uint16_t bodyLength, uint8_t *response)</code>

Service Name	relayCommand
Parameters (in)	header: an already-serialised command header, PP_CMD_HEADER_SIZE bytes. body: already-serialised body, or nullptr when empty. bodyLength: bytes in body.
Parameters (inout)	None
Parameters (out)	response: receives PP_RESPONSE_SIZE bytes.
Return value	false if the arguments are unusable or no context is active. A board answering with an ERROR status still returns true: that is a reply, and only the end that composed the command can judge it.
Description	Shape 1. Sends the header, sends the body in the board's own chunks, then polls until the board answers or the response deadline expires.
Available via	PAPAYA.h (C++)

3.8.4 relayCommandWithData

[PPL_API_731]:

Service Name	relayCommandWithData
Syntax (C++)	bool relayCommandWithData(const uint8_t *header, const uint8_t *body, uint16_t bodyLength, uint8_t *response, uint8_t *data, uint16_t dataLength)
Parameters (in)	As relayCommand, plus dataLength: how many payload bytes to expect.
Parameters (inout)	None
Parameters (out)	response, and data: receives dataLength bytes.
Return value	false when the command failed, or when the board's status reported an error.
Description	Shape 2. The payload is read ONLY after a status that reported success. Reading one the board never queued consumes the next command's response instead, and the link is out of step from that point on.
Available via	PAPAYA.h (C++)

3.8.5 relayCommandNoResponse

[PPL_API_732]:

Service Name	relayCommandNoResponse
Syntax (C++)	<code>void relayCommandNoResponse(const uint8_t *header, const uint8_t *body, uint16_t bodyLength)</code>
Parameters (in)	As relayCommand.
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Shape 5. Sends the command and does not wait. For the commands that restart the board: it is gone before it could compose an answer. Which commands those are is the composing end's knowledge, not this library's.
Available via	PAPAYA.h (C++)

3.8.6 relayBeginCommand

[PPL_API_733]:

Service Name	relayBeginCommand
Syntax (C++)	<code>void relayBeginCommand(const uint8_t *header)</code>
Parameters (in)	header: an already-serialised command header, PP_CMD_HEADER_SIZE bytes.
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Shape 1, first of three steps. Puts the header on the wire and takes the sequence number out of it, so the poll built later carries the number the board will answer with. Use this form when your host receives a command a piece at a time and never holds the whole body: the largest legitimate command carries an inline table of about 64 KB, and a host with 32 KB of memory can still relay one. After this you MUST send the whole declared body and then poll, even if that means sending filler.

Service Name	relayBeginCommand
Available via	PAPAYA.h (C++)

3.8.7 relaySendBodyChunk

[PPL_API_734]:

Service Name	relaySendBodyChunk
Syntax (C++)	bool relaySendBodyChunk(const uint8_t *chunk, uint16_t length)
Parameters (in)	chunk: body bytes. length: at most PP_CMD_CHUNK_SIZE, which is the board's own figure and not a tuning choice.
Parameters (inout)	None
Parameters (out)	None
Return value	false if the arguments are unusable or no context is active.
Description	Shape 1, second of three. One piece of the body of a command already begun. Call as many times as the body needs.
Available via	PAPAYA.h (C++)

3.8.8 relayAwaitResponse

[PPL_API_735]:

Service Name	relayAwaitResponse
Syntax (C++)	bool relayAwaitResponse(uint8_t *response)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	response: receives PP_RESPONSE_SIZE bytes.
Return value	false only when the conversation did not happen. An error status is a reply.
Description	Shape 1, last of three. Polls for the status of a command already sent whole. Until the board has an answer ready it returns the request unchanged, so the reply is checked against its own checksum rather than trusted for arriving.
Available via	PAPAYA.h (C++)

3.8.9 relayReadDataPayload

[PPL_API_736]:

Service Name	relayReadDataPayload
Syntax (C++)	<code>bool relayReadDataPayload(uint8_t *data, uint16_t dataLength)</code>
Parameters (in)	dataLength: how many bytes to expect.
Parameters (inout)	None
Parameters (out)	data: receives dataLength bytes.
Return value	false when the read did not happen.
Description	Shape 2, as a step of its own. Call ONLY after a status that reported success. Reading a payload the board never queued consumes the next command's response instead.
Available via	PAPAYA.h (C++)

3.8.10 relayReadProbes

[PPL_API_737]:

Service Name	relayReadProbes
Syntax (C++)	<code>uint16_t relayReadProbes(uint8_t *buffer, uint16_t byteCount)</code>
Parameters (in)	byteCount: probes times samples times sizeof(float), from the end that knows the design.
Parameters (inout)	None
Parameters (out)	buffer: receives up to byteCount bytes.
Return value	Bytes read, or 0.
Description	Shape 3. The samples arrive probe-major: every sample of the first probe, then every sample of the second.
Available via	PAPAYA.h (C++)

3.8.11 relayReadVectorProbes

[PPL_API_738]:

Service Name	relayReadVectorProbes
Syntax (C++)	<code>uint16_t relayReadVectorProbes(uint8_t *buffer, uint16_t byteCount)</code>
Parameters (in)	byteCount: total bytes of every vector probe.
Parameters (inout)	None
Parameters (out)	buffer: receives up to byteCount bytes.
Return value	Bytes read, or 0.
Description	Shape 3 for vector probes. Same contract as relayReadProbes.
Available via	PAPAYA.h (C++)

3.8.12 relayWriteInputs

[PPL_API_739]:

Service Name	relayWriteInputs
Syntax (C++)	<code>bool relayWriteInputs(const uint8_t *values, uint16_t byteCount)</code>
Parameters (in)	values: the input values as bytes. byteCount: their total size.
Parameters (inout)	None
Parameters (out)	None
Return value	false when the write did not happen.
Description	Shape 4. Every value travels in ONE exchange. Splitting them leaves the firmware waiting for the rest and mis-reading whatever arrives next as input data.
Available via	PAPAYA.h (C++)

3.8.13 resetBoard

[PPL_API_740]:

Service Name	resetBoard
Syntax (C++)	<code>bool resetBoard(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None

Service Name	resetBoard
Return value	false when the active transport has no reset hook, so you can tell "not wired" from "tried and failed".
Description	Assert and release the board's reset line, using the same assert and boot-settle timing as start-up, returning once the board has had time to come back. The capability a USB host does not have: over a cable a host can only ASK the board to reset itself, which is no help once it has stopped listening. A wired host owns the line.
Available via	PAPAYA.h (C++)

3.9 Other services

Complete for reference; the groups above cover ordinary use.

Service	Python	C++
eraseAudioSlot	yes	yes
eraseDataset	yes	yes
getAudioSlotInfo	yes	yes
getBootCfg	yes	yes
getDatasetInfo	yes	yes
getDesignInfo	yes	yes
listAudioSlots	yes	yes
listDatasets	yes	yes
loadApp	yes	yes
mcuReset	yes	yes
readProbeLive	yes	-
readProbes	yes	-
readProbesLatest	yes	yes
readProbesLive	yes	yes
readVectProbeLive	yes	-
readVectProbes	yes	-
readVectProbesLive	yes	-
remoteProbeIndex	-	yes
setDwtDenoiseScaleLive	yes	-

Service	Python	C++
setDwtSubbandGainsLive	yes	-
setFreqInCh2Source	yes	yes
setFreqInSwapPins	yes	yes
setInput	yes	-
setInputsLive	yes	-
signalSourceManualTrigger	yes	yes
softReset	yes	yes
switchToCfg	yes	-
syncEeprom	yes	yes
uploadApp	yes	yes
uploadAudioSlotChunk	yes	yes
uploadAudioSlotCommit	yes	yes
uploadAudioSlotStart	yes	yes
uploadDatasetChunk	yes	yes
uploadDatasetCommit	yes	yes
uploadDatasetStart	yes	yes
writeInputs	yes	-
writeInputsLive	yes	yes

Chapter 4

Constructing blocks

Every block is a class. Construct an instance, then link it.

Note

What each block **computes** - ports, mathematics, validity - is in *PAPAYA Studio – User Manual*. This section is the calling convention only.

4.1 Transfer functions (82)

4.1.1 CustFir

Custom FIR filter with user-provided tap coefficients.

```
CustFir(coeff: 'Sequence[float]', taps: 'int') -> 'None'
```

```
CustFir(float *coefficients, uint16_t size)
```

Block type CUSTOM_FIR.

4.1.2 UpDwnperiod

Up/Down period measurement - measures the high-time (up) and low-time (down) of a signal separately.

```
UpDwnperiod(Max: 'float', Min: 'float', Epsilon: 'float') -> 'None'
```

```
UpDwnperiod(float Max, float Min, float Epsilon)
```

Block type UP_DOWN_PERIOD.

4.1.3 absMax

Sliding-window absolute maximum over the last N samples.

```
absMax(Length: 'int') -> 'None'
```

```
absMax(uint8_t Length)
```

Block type ABS_MAX.

4.1.4 absMin

Sliding-window absolute minimum over the last N samples.

```
absMin(Length: 'int') -> 'None'
```

```
absMin(uint8_t Length)
```

Block type ABS_MIN.

4.1.5 absVal

Absolute value function - returns the magnitude of the input regardless of sign.

```
absVal() -> 'None'
```

```
absVal(void)
```

Block type ABS_VAL.

4.1.6 adsr

ADSR envelope generator driven by a gate input.

```
adsr(Attack_s: 'float', Decay_s: 'float', Sustain: 'float', Release_s: 'float') -> 'None'
```

```
adsr(float Attack_s, float Decay_s, float Sustain, float Release_s)
```

Block type ADSR.

4.1.7 allpass

All-pass filter: unity magnitude at every frequency, with a phase shift you choose at a reference frequency you choose.

```
allpass(val1: 'float', val2: 'float') -> 'None'
```

```
allpass(float freq, float phi)
```

Block type ALL_PASS.

4.1.8 amOscillator

Amplitude-modulated periodic oscillator - a 128-point lookup-table waveform whose output is multiplied at every sample by an external amplitude input.

```
amOscillator(waveform: 'PP_WaveForm', freq: 'float', phaseshift: 'float') -> 'None'
amOscillator(PP_WaveForm waveform, float freq, float phaseshift)
```

Block type AM_OSCILLATOR.

4.1.9 autocorrLags

Lag-windowed autocorrelation, decimated refresh.

```
autocorrLags(N: 'int' = 256, L: 'int' = 64, D: 'int' = 32) -> 'None'
autocorrLags(uint16_t N, uint16_t L, uint16_t D)
```

Block type AUTOCORR_LAGS.

4.1.10 bitcrusher

Bitcrusher - bit-depth reduction combined with sample-rate decimation, for lo-fi / retro-digital textures.

```
bitcrusher(BitDepth: 'float', SRReduction: 'int') -> 'None'
bitcrusher(float BitDepth, uint16_t SRReduction)
```

Block type BITCRUSHER.

4.1.11 butter

Butterworth IIR filter with maximally-flat magnitude response.

```
butter(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType' =
  ↳ <PP_FilterType.NoFilterType: 255>) -> 'None'
butter(uint8_t n, float fc, PP_FilterType ftype)
butter(uint8_t n, float fc1, float fc2, PP_FilterType ftype)
```

Block type BUTTERWORTH.

4.1.12 cheby1

Chebyshev Type I IIR filter expressed as a transfer function (TF).

```
cheby1(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType' =
  ↳ <PP_FilterType.NoFilterType: 255>, chebyCfg: 'int' = 255) -> 'None'
cheby1(uint8_t n, float fc, PP_FilterType ftype, PP_Cheby1Cfg chebyCfg)
cheby1(uint8_t n, float fc1, float fc2, PP_FilterType ftype, PP_Cheby1Cfg chebyCfg)
```

Block type CHEBYSHEV1.

4.1.13 cheby2

Chebyshev Type II (inverse Chebyshev) IIR filter expressed as a transfer function (TF).

```
cheby2(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType' =  
↪ <PP_FilterType.NoFilterType: 255>, chebyCfg: 'int' = 255) -> 'None'
```

```
cheby2(uint8_t n, float fc, PP_FilterType ftype, PP_Cheby2Cfg chebyCfg)  
cheby2(uint8_t n, float fc1, float fc2, PP_FilterType ftype, PP_Cheby2Cfg chebyCfg)
```

Block type CHEBYSHEV2.

4.1.14 chorus

Chorus effect - an LFO-modulated short delay line summed with the dry signal.

```
chorus(Rate_Hz: 'float', Depth_ms: 'float', Mix: 'float') -> 'None'
```

```
chorus(float Rate_Hz, float Depth_ms, float Mix)
```

Block type CHORUS.

4.1.15 combFeedBack

Comb filter (feed-back) - creates a series of evenly spaced resonances in the frequency response.

```
combFeedBack(delay: 'int', gain: 'float') -> 'None'
```

```
combFeedBack(uint16_t delay, float gain)
```

Block type COMB_FB.

4.1.16 combFeedForward

Comb filter (feed-forward) - creates a series of evenly spaced notches in the frequency response.

```
combFeedForward(delay: 'int', gain: 'float') -> 'None'
```

```
combFeedForward(uint16_t delay, float gain)
```

Block type COMB_FF.

4.1.17 compressor

Dynamics compressor - reduces the dynamic range of a signal by attenuating levels above a threshold.

```
compressor(attack: 'float', release: 'float', threshold_db: 'float', ratio: 'float') -> 'None'
```

```
compressor(float Attack, float Release, float Threshold_dB, float Ratio)
```

Block type COMPRESSOR.

4.1.18 cosineFunc

Cosine function - computes $\cos(x)$ where x is in radians.

```
cosineFunc() -> 'None'
```

```
cosineFunc(void)
```

Block type COSINE_FUNC.

4.1.19 cpuLoad

CPU load measurement block.

```
cpuLoad() -> 'None'
```

```
cpuLoad()
```

Block type CPU_LOAD.

4.1.20 crestFactor

Crest factor: ratio of peak amplitude to RMS over a sliding window.

```
crestFactor(window_N: 'int' = 64) -> 'None'
```

```
crestFactor(uint16_t WindowN)
```

Block type CREST_FACTOR.

4.1.21 tf

Continuous-time linear transfer function, given as numerator and denominator coefficients in descending powers of s . **This is the block the whole continuous-time workflow rests on:** you describe the plant or the compensator in the Laplace domain and the board discretises it, using the design's sampling frequency and its continuous-to-discrete algorithm, at the moment the block is created.

```
tf(numerator=None, denominator=None, numSize=None, denomSize=None) -> None
```

```
tf()
```

```
tf(const char *numerator, const char *denominator)
```

```
tf(float *numerator, uint8_t numSize, float *denominator, uint8_t denomSize)
```

Both libraries accept the coefficients either as a string, which is parsed, or as an array with an explicit length.

The sampling frequency and the algorithm are read when the block is created, not when the design starts. Call `startDesign` with the frequency you mean before constructing any continuous block, or the coefficients will be discretised against the wrong rate.

Block type TRANSFER_FUNCTION.

4.1.22 node

Generic summing and routing node with configurable input and output coefficients. A node is what forms an error signal, adds two signals, scales one, or fans a value into several consumers.

```
node(inCoef=None, outCoef=None, Type=None) -> None

node()
node(const char *inCoef, const char *outCoef, PP_NodeType Type)
node(const float *inCoef, uint8_t inSize,
      const float *outCoef, uint8_t outSize, PP_NodeType Type)
```

Type is a PP_NodeType. The coefficient arrays give the weight applied to each input and each output, so a two-input node with input coefficients 1, -1 is a subtractor.

A node is the block class a feedback loop must be closed onto. The board refuses to start a transfer-function block whose input is still unwired, so the loop-closing link of a cycle has to target a node-class block rather than an integrator or a delay.

Block type NODE.

4.1.23 macro

Composite subgraph: a block that contains a diagram of its own, with a declared number of input and output ports.

```
macro(nbrOfInputs=None, nbrOfOutputs=None) -> None

macro()
macro(uint8_t NbrOfInputs, uint8_t NbrOfOutputs)
```

The port counts are the macro's boundary. Blocks inside it are created and linked exactly as they are at the top level.

Probes are refused inside a macro. A nested probe takes an earlier monitor slot and mislabels every probe column, so probes belong at the top level. Remote-enabled inputs, by contrast, are counted through macros and share the one budget of 16.

Block type MACRO.

4.1.24 delayop

Pure delay operator - delays the input signal by N samples.

```
delayop(NbrOfSamples: 'int') -> 'None'

delayop(uint16_t NbrOfSamples)
```

In C++ the class is delay.

Block type DELAY.

4.1.25 derivativeBlock

Filtered derivative - computes the rate of change of the input with high-frequency roll-off.

```
derivativeBlock(tau: 'float') -> 'None'
```

```
derivative(float Tau = 0.01f)
```

In C++ the class is derivative.

Block type DERIVATIVE.

4.1.26 distortion

Waveshaping distortion with selectable nonlinearity.

```
distortion(Drive: 'float', Mix: 'float', Type: 'int') -> 'None'
```

```
distortion(float Drive, float Mix, uint8_t Type)
```

Block type DISTORTION.

4.1.27 dwt

Discrete Wavelet Transform (DWT) - decomposes a signal into time-frequency components at multiple resolution levels.

```
dwt(signal_length: 'int', wavelet_type: 'PP_WaveletType', levels: 'int', sampling_rate:
    ↪ 'float') -> 'None'
```

```
dwt(uint16_t signal_length, PP_WaveletType wavelet_type, uint8_t levels, float sampling_rate)
dwt()
```

Block type DWT.

4.1.28 dwt_denoise

Wavelet Denoising block - estimates noise via MAD from the finest detail subband, then applies level-dependent thresholding to all detail subbands.

```
dwt_denoise(associated_dwt: 'dwt', thresh_type: 'PP_ThresholdType' =
    ↪ <PP_ThresholdType.PP_THRESH_SOFT: 1>, thresh_scale: 'float' = 1.0) -> 'None'
```

```
dwt_denoise(dwt &dwtInstance, PP_ThresholdType thresh_type = PP_THRESH_SOFT, float
    ↪ thresh_scale = 1.0f)
```

Block type DWT_DENOISE.

4.1.29 dwt_subband_energy

Subband energy measurement - computes the energy (sum of squared coefficients) of each wavelet subband.

```
dwt_subband_energy(associated_dwt: 'dwt') -> 'None'
```

```
dwt_subband_energy(dwt &dwtInstance)
```

Block type DWT_SUBBAND_ENERGY.

4.1.30 dwt_subband_gain

Per-subband gain control - applies a scalar gain to each wavelet subband.

```
dwt_subband_gain(associated_dwt: 'dwt', gains: 'Optional[Sequence[float]]' = None) -> 'None'
dwt_subband_gain(dwt &dwtInstance, const float* gains = nullptr)
```

Block type DWT_SUBBAND_GAIN.

4.1.31 ecgBlRemove

ECG baseline-wander remover.

```
ecgBlRemove(window_W: 'int' = 50) -> 'None'
ecgBlRemove(uint16_t WindowW)
```

Block type ECG_BL_REMOVE.

4.1.32 ellip

Elliptic (Cauer) IIR filter expressed as a transfer function (TF).

```
ellip(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType' =
↳ <PP_FilterType.NoFilterType: 255>, ellipCfg: 'int' = 255) -> 'None'
ellip(uint8_t n, float fc, PP_FilterType ftype, PP_EllipCfg ellipCfg)
ellip(uint8_t n, float fc1, float fc2, PP_FilterType ftype, PP_EllipCfg ellipCfg)
```

Block type ELLIPTIC.

4.1.33 emgEnvelope

EMG envelope detector.

```
emgEnvelope(cutoff_hz: 'float' = 5.0) -> 'None'
emgEnvelope(float CutoffHz)
```

Block type EMG_ENVELOPE.

4.1.34 entropy

Sliding-window Shannon entropy treating the raw window samples as a probability vector.

```
entropy(Length: 'int') -> 'None'
entropy(uint8_t Length)
```

Block type ENTROPY.

4.1.35 envFollower

Envelope follower with asymmetric attack/release smoothing on $|x|$.

```
envFollower(Attack_s: 'float', Release_s: 'float') -> 'None'
```

```
envFollower(float Attack_s, float Release_s)
```

Block type ENV_FOLLOWER.

4.1.36 expFunc

Exponential function - computes the natural exponential of the input.

```
expFunc() -> 'None'
```

```
expFunc(void)
```

Block type EXP_FUNC.

4.1.37 fft

Fast Fourier Transform (FFT) - computes the frequency spectrum of a time-domain signal.

```
fft(NbSamples: 'int') -> 'None'
```

```
fft(uint16_t Nbsamples)
```

Block type FFT.

4.1.38 fftMagResolver

Magnitude spectrum resolver - extracts the magnitude at each frequency bin from an FFT output.

```
fftMagResolver(fft_instance: 'Union[fft, fft_HannWindow]') -> 'None'
```

```
fftMagResolver(fft fftInstance)
```

```
fftMagResolver(fft_HannWindow fft_hann_Instance)
```

Block type FFT_MAG.

The published vector holds $N/2 + 1$ magnitudes for a transform of length N , and it is not in ascending frequency order. The layout is packed:

index 0	magnitude of bin 0	0 Hz (DC)
index 1	magnitude of bin $N/2$	$F_s/2$ (Nyquist)
index $k+1$	magnitude of bin k	$k \cdot F_s/N$ Hz ($k = 1 \dots N/2-1$)

To build a frequency axis over the raw array, map index 0 to 0 Hz, index 1 to $F_s/2$, and every index $m \geq 2$ to $(m-1) \cdot F_s/N$. In the Studio, the vector tile's Magnitude (dB) and Magnitude (linear) plot views already reorder the spectrum into natural frequency order; the Table view, and a CSV exported while it is active, show the raw wire order above. A CSV exported from a plot view carries the plotted, frequency-ordered data.

4.1.39 fft_HannWindow

FFT with Hann window applied before the transform to reduce spectral leakage.

```
fft_HannWindow(NbSamples: 'int') -> 'None'
```

```
fft_HannWindow(uint16_t Nbsamples)
```

Block type FFT_HANN.

4.1.40 fft_phaseResolver

Phase spectrum resolver - extracts the phase angle at each frequency bin from an FFT output.

```
fft_phaseResolver(fft_instance: 'Union[fft, fft_HannWindow]') -> 'None'
```

```
fft_phaseResolver(fft fftInstance)
```

```
fft_phaseResolver(fft_HannWindow fft_hann_Instance)
```

Block type FFT_PHASE.

The phase vector uses the same packed layout as fftMagResolver: index 0 is the DC bin and index 1 the Nyquist bin, both fixed at 0 by convention, and index k+1 holds the phase of bin k, in radians in $(-\pi, +\pi]$. Convert an index $m \geq 2$ to frequency with $(m-1) \cdot F_s/N$. The Studio's Phase (wrapped, deg) and Phase (unwrapped, deg) views reorder into natural frequency order and convert to degrees; the Table view and its CSV export show the raw wire order in radians.

4.1.41 fir

Finite Impulse Response (FIR) filter designed using the windowed-sinc method.

```
fir(N: 'int', f1: 'float', f2: 'Optional[float]' = None, wt: 'PP_FirWindowType' =
```

```
↳ <PP_FirWindowType.NoFirWindowType: 255>, ftype: 'PP_FilterType' =
```

```
↳ <PP_FilterType.NoFilterType: 255>) -> 'None'
```

```
fir(uint16_t N, float f1, PP_FirWindowType wt, PP_FilterType ftype)
```

```
fir(uint16_t N, float f1, float f2, PP_FirWindowType wt, PP_FilterType ftype)
```

Block type FIR.

4.1.42 flanger

Flanger effect - a very short LFO-modulated delay with feedback, producing the characteristic sweeping comb-filter sound.

```
flanger(Rate_Hz: 'float', Depth_ms: 'float', Feedback: 'float', Mix: 'float') -> 'None'
```

```
flanger(float Rate_Hz, float Depth_ms, float Feedback, float Mix)
```

Block type FLANGER.

4.1.43 fracDelay

Fractional (linearly interpolated) delay line with a fixed, construct-time delay length.

```
fracDelay(MaxDelaySamples: 'int') -> 'None'
```

```
fracDelay(uint32_t MaxDelaySamples)
```

Block type FRAC_DELAY.

4.1.44 frictionStribeck

Static Stribeck friction feedforward.

```
frictionStribeck(Fc: 'float', Fs: 'float', vs: 'float', Bv: 'float') -> 'None'
```

```
frictionStribeck(float Fc, float Fs, float vs, float Bv)
```

Block type FRICTION_STRIBECK.

4.1.45 idwt

Inverse Discrete Wavelet Transform (IDWT) - reconstructs a time-domain signal from DWT coefficients.

```
idwt(associated_dwt: 'dwt') -> 'None'
```

```
idwt(dwt &dwtInstance)
```

Block type IDWT.

4.1.46 inputShaper

Convolutional input shaper for vibration suppression.

```
inputShaper(mode: 'int' = 0, wn: 'float' = 10.0, zeta: 'float' = 0.05) -> 'None'
```

```
inputShaper(uint8_t Mode, float Wn, float Zeta)
```

Block type INPUT_SHAPER.

4.1.47 integratorBlock

Pure integrator - accumulates the input signal over time.

```
integratorBlock() -> 'None'
```

```
integrator(void)
```

In C++ the class is integrator.

Block type INTEGRATOR.

4.1.48 interpolation

Interpolator block - the anti-imaging low-pass filter of an L-fold interpolation chain.

```
interpolation(M_u32: 'int') -> 'None'
```

```
interpolation(uint32_t M_u32)
```

Block type INTERPOLATION.

4.1.49 inverseBlock

Reciprocal (inverse) function - computes the multiplicative inverse of the input.

```
inverseBlock() -> 'None'
```

```
inverse(void)
```

In C++ the class is inverse.

Block type INVERSE.

4.1.50 leadLag

Lead-lag compensator for frequency-domain loop shaping.

```
leadLag(K: 'float', z: 'float', p: 'float') -> 'None'
```

```
leadLag(float K, float z, float p)
```

Block type LEAD_LAG.

4.1.51 limiter

Peak limiter with attack/release smoothing and a dB ceiling.

```
limiter(Attack_s: 'float', Release_s: 'float', Ceiling_dB: 'float') -> 'None'
```

```
limiter(float Attack_s, float Release_s, float Ceiling_dB)
```

Block type LIMITER.

4.1.52 logFunc

Natural logarithm (base e) function.

```
logFunc() -> 'None'
```

```
logFunc(void)
```

Block type LOG_FUNC.

4.1.53 maxObserver

Maximum observer block.

```
maxObserver(window_size: 'int') -> 'None'
```

```
maxObserver(uint8_t windowSize)
```

Block type MAX_OBSERVER.

4.1.54 maximum

Sliding-window maximum over the last N samples.

```
maximum(Length: 'int') -> 'None'
```

```
maximum(uint32_t Length)
```

Block type MAXIMUM.

4.1.55 mean

Sliding-window arithmetic mean (average) over the last N samples.

```
mean(Length: 'int') -> 'None'
```

```
mean(uint8_t Length)
```

Block type MEAN.

4.1.56 minObserver

Minimum observer block.

```
minObserver(window_size: 'int') -> 'None'
```

```
minObserver(uint8_t windowSize)
```

Block type MIN_OBSERVER.

4.1.57 minimum

Sliding-window minimum over the last N samples.

```
minimum(Length: 'int') -> 'None'
```

```
minimum(uint32_t Length)
```

Block type MINIMUM.

4.1.58 momentaryPower

Momentary power measurement block.

```
momentaryPower(block_size: 'int') -> 'None'
```

```
momentaryPower(uint16_t windowSize)
```

Block type MOMENTARY_POWER.

4.1.59 movingAverage

Moving average filter - computes the unweighted mean of the last N samples.

```
movingAverage(NbrOfSamples: 'int') -> 'None'
```

```
movingAverage(uint16_t NbrOfSamples)
```

Block type MOVING_AVERAGE.

4.1.60 negateBlock

Negation function - inverts the sign of the input.

```
negateBlock() -> 'None'
```

```
negate(void)
```

In C++ the class is negate.

Block type NEGATE.

4.1.61 noise

Random noise generator supporting uniform and Gaussian distributions.

```
noise(value1: 'float', value2: 'float', ntype: 'PP_NoiseType') -> 'None'
```

```
noise(float value1, float value2, PP_NoiseType type)
```

Block type NOISE.

4.1.62 noiseGate

Downward expander / noise gate with attack/release smoothing.

```
noiseGate(Attack_s: 'float', Release_s: 'float', Threshold_dB: 'float', Range_dB: 'float') ->  
↳ 'None'
```

```
noiseGate(float Attack_s, float Release_s, float Threshold_dB, float Range_dB)
```

Block type NOISE_GATE.

4.1.63 notch

Notch (band-reject) filter that removes a narrow frequency band around a center frequency.

```
notch(centernotch: 'float', qualcoef: 'float') -> 'None'
```

```
notch(float centernotch, float qualcoef)
```

Block type NOTCH.

4.1.64 peak

Peaking (band-pass resonator) filter that boosts a narrow frequency band around a center frequency.

```
peak(center_freq: 'float', quality_factor: 'float') -> 'None'
peak(float centernotch, float qualcoef)
```

Block type PEAK.

4.1.65 peakToPeak

Sliding-window peak-to-peak amplitude.

```
peakToPeak(window_N: 'int' = 64) -> 'None'
peakToPeak(uint16_t WindowN)
```

Block type PEAK_TO_PEAK.

4.1.66 percentile

Jain-Chlamtac P² streaming percentile.

```
percentile(quantile: 'float' = 0.5) -> 'None'
percentile(float Quantile)
```

Block type PERCENTILE.

4.1.67 period

Period measurement - measures the signal period by timing the interval between consecutive rising (or falling) edges.

```
period(Max: 'float', Min: 'float', Epsilon: 'float') -> 'None'
period(float Max, float Min, float Epsilon)
```

Block type PERIOD.

4.1.68 phaser

Phaser effect - cascaded allpass stages with LFO-modulated coefficients, summed with the dry signal to produce moving notches.

```
phaser(Rate_Hz: 'float', Depth: 'float', Feedback: 'float', NumStages: 'int', Mix: 'float')
↳ -> 'None'
phaser(float Rate_Hz, float Depth, float Feedback, uint8_t NumStages, float Mix)
```

Block type PHASER.

4.1.69 pid

Proportional-Integral-Derivative (PID) controller.

```
pid(ki: 'float', kp: 'float', kd: 'float', tau: 'float') -> 'None'
pid(float ki, float kp)
pid(float kp, float kd, float tau)
pid(float ki, float kp, float kd, float tau)
```

Block type PID.

4.1.70 pitchDetect

Autocorrelation-based fundamental-frequency detector (YIN).

```
pitchDetect(MinFreq: 'float', MaxFreq: 'float') -> 'None'
pitchDetect(float MinFreq, float MaxFreq)
```

Block type PITCH_DETECT.

4.1.71 power

Sliding-window signal power (sum of squares) over the last N samples.

```
power(Length: 'int') -> 'None'
power(uint8_t Length)
```

Block type POWER.

4.1.72 reverb

Feedback-delay-network (FDN) reverberator - simulates room reverberation with four non-harmonic delay lines and a diffusing feedback matrix.

```
reverb(DecayTime_s: 'float', Mix: 'float') -> 'None'
reverb(float DecayTime_s, float Mix)
```

Block type REVERB.

4.1.73 rootMeanSquare

Sliding-window Root Mean Square (RMS) over the last N samples.

```
rootMeanSquare(Length: 'int') -> 'None'
rootMeanSquare(uint8_t Length)
```

Block type RMS.

4.1.74 signalSource

Unified signal generator - every modern waveform shape (Sine, Square, Triangle, Sawtooth, Pulse, Sinc, Gauspuls, DC, Arbitrary LUT, Linear/Log Chirp, .

```

signalSource(shape: 'PP_WaveShape' = <PP_WaveShape.Sine: 0>, mode: 'PP_WaveMode' =
↳ <PP_WaveMode.Perpetual: 0>, n_periods: 'int' = 0, trig_pol: 'PP_WaveTrigPol' =
↳ <PP_WaveTrigPol.Rising: 0>, amplitude: 'float' = 1.0, offset: 'float' = 0.0,
↳ **shape_kwargs) -> 'None'

```

```

signalSource(WaveShape shape, float amplitude = 1.0f, float offset = 0.0f, WaveMode mode =
↳ WaveMode::Perpetual, uint32_t n_periods = 0, WaveTrigPol trig_pol = WaveTrigPol::Rising,
↳ bool emitFrame = true)

```

Block type SIGNAL_SOURCE.

4.1.75 sineFunc

Sine function - computes $\sin(x)$ where x is in radians.

```

sineFunc() -> 'None'

```

```

sineFunc(void)

```

Block type SINE_FUNC.

4.1.76 sqrMath

Square function - computes the square of the input.

```

sqrMath() -> 'None'

```

```

sqrMath(void)

```

Block type SQR_MATH.

4.1.77 sqrtBlock

Square root function - computes the principal square root of the input.

```

sqrtBlock() -> 'None'

```

```

sqrtBlock(void)

```

Block type SQRT.

4.1.78 standardDeviation

Sliding-window standard deviation over the last N samples.

```

standardDeviation(Length: 'int') -> 'None'

```

```

standardDeviation(uint8_t Length)

```

Block type STD_DEV.

4.1.79 stftFramer

STFT framer - sliding-window framer with analysis window applied.

```
stftFramer(N: 'int', H: 'int', stft_window: 'int' = 0) -> 'None'
stftFramer(uint16_t N, uint16_t H, uint16_t WindowType)
```

Block type STFT_FRAMER.

4.1.80 tfd

Discrete-time linear transfer function with user-provided z-domain coefficients.

```
tfd(numerator: 'Sequence[float]', denominator: 'Sequence[float]', sysord: 'int') -> 'None'
tfd(float *numerator, float *denominator, uint8_t sysord)
```

Block type TRANSFER_FUNCTION_DISCRETE.

4.1.81 variance

Sliding-window variance over the last N samples.

```
variance(Length: 'int') -> 'None'
variance(uint8_t Length)
```

Block type VARIANCE.

4.1.82 vuMeter

VU / PPM meter - asymmetric rise/fall smoothing on $|x|$, output in dB.

```
vuMeter(RiseTime_s: 'float', FallTime_s: 'float') -> 'None'
vuMeter(float RiseTime_s, float FallTime_s)
```

Block type VU_METER.

4.1.83 washout

Washout (high-pass) filter that removes the DC component while passing transients.

```
washout(a: 'float') -> 'None'
washout(float a)
```

Block type WASHOUT.

4.1.84 windowFramer

Sliding-window framer with hop H.

```
windowFramer(N: 'int', H: 'int') -> 'None'
windowFramer(uint16_t N, uint16_t H)
```

Block type WINDOW_FRAMER.

4.1.85 zcr

Zero-crossing rate over a block window.

```
zcr(window_N: 'int' = 256, zc_threshold: 'float' = 0.0) -> 'None'
zcr(uint16_t WindowN, float ZcThreshold)
```

Block type ZCR.

4.2 Nodes (114)

4.2.1 RLS

Recursive Least Squares (RLS) for online system identification.

```
RLS(data: 'PP_RLSParam') -> 'None'
RLS(PP_RLSParam *data)
```

Block type RLS.

4.2.2 RRLS

Recursive Least Squares with an error-dependent forgetting factor, for identifying a plant whose dynamics change during the run.

```
RRLS(data: 'PP_RRLSParam') -> 'None'
RRLS(PP_RRLSParam *data)
```

Block type RRLS.

4.2.3 accelerometer

```
accelerometer(outScale: 'float') -> 'None'
imu(PP_HwId id, float outScale)
```

In C++ the class is imu.

4.2.4 acimEKF

Extended Kalman Filter for AC Induction Motor (ACIM) state estimation.

```
acimEKF(config: 'PP_ACIM_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
acimEKF(PP_ACIM_EKF *config)
```

Block type ACIM_EKF.

4.2.5 adaptiveKF

Adaptive Kalman Filter (AKF) for state estimation in systems where the measurement noise characteristics change over time.

```
adaptiveKF(AKF: 'PP_adaptiveKF', select: 'Optional[PP_Tune]' = None) -> 'None'
```

```
adaptiveKF(PP_adaptiveKF *AKF)
adaptiveKF(PP_adaptiveKF *AKF, PP_Tune select)
```

Block type ADAPTIVE_KF.

4.2.6 adaptiveNotch

Adaptive notch filter that tracks and removes a time-varying sinusoidal interference.

```
adaptiveNotch(config: 'PP_AdaptiveNotchConfig') -> 'None'
```

```
adaptiveNotch(PP_AdaptiveNotchConfig *config)
```

Block type ADAPTIVE_NOTCH.

4.2.7 adrc

Active Disturbance Rejection Control (ADRC).

```
adrc(config: 'PP_ADRCConfig') -> 'None'
```

```
adrc(PP_ADRCConfig *config)
```

Block type ADRC.

4.2.8 ahrsEKF

Extended Kalman Filter for attitude and heading (AHRS) estimation.

```
ahrsEKF(config: 'PP_AHRS_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
```

```
ahrsEKF(PP_AHRS_EKF *config)
```

Block type AHRS_EKF.

4.2.9 analogIn

Analog input block that reads one of the board's analog input channels in volts and optionally converts to engineering units.

```
analogIn(id_or_cfg: 'Union[PP_HwId, Sequence[PP_anInCfg]]', outScale: 'Optional[float]' =  
↳ None, offset: 'Optional[float]' = None, items: 'Optional[int]' = None) -> 'None'
```

```
analogIn(PP_HwId id, float outScale, float offset)  
analogIn(PP_anInCfg *config, uint8_t items)
```

Block type ANALOG_IN.

4.2.10 analogOut

Analog voltage output block driving the board's analog output channels.

```
analogOut(id_or_cfg: 'Union[PP_HwId, Sequence[PP_anOutCfg], dict, None]' = None, lowerLimit:
↳ 'Optional[float]' = None, upperLimit: 'Optional[float]' = None, items: 'Optional[int]' =
↳ None, *, mode: 'Union[PP_AnOutMode, int, str]' = <PP_AnOutMode.InputDriven: 0>, shape:
↳ 'Optional[PP_WaveShape]' = None, amplitude: 'float' = 1.0, offset: 'float' = 0.0,
↳ **shape_kwargs) -> 'None'
```

```
analogOut(PP_HwId id, float lowerLimit, float upperLimit)
analogOut(PP_anOutCfg *config, uint8_t items)
```

Block type ANALOG_OUT.

4.2.11 atan2Block

Two-argument arc-tangent - computes the angle of the vector (x, y) in the full four-quadrant plane.

```
atan2Block() -> 'None'
```

```
atan2Block(void)
```

Block type ATAN2.

4.2.12 backlashDeadBand

Memoryless dead-zone with offset compensation.

```
backlashDeadBand() -> 'None'
```

```
backlashDeadBand()
```

Block type BACKLASH.

4.2.13 baroAltEKF

Extended Kalman Filter for barometric altitude estimation.

```
baroAltEKF(config: 'PP_BaroAlt_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
```

```
baroAltEKF(PP_BaroAlt_EKF *config)
```

Block type BARO_ALT_EKF.

4.2.14 batterySOC_EKF

Extended Kalman Filter for battery state-of-charge estimation.

```
batterySOC_EKF(config: 'PP_BatterySOC_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) ->
↳ 'None'
```

```
batterySOC_EKF(PP_BatterySOC_EKF *config)
```

Block type BATTERY_SOC_EKF.

4.2.15 batterySOH_EKF

Extended Kalman Filter for battery State of Health (SOH) estimation.

```
batterySOH_EKF(config: 'PP_BatterySOH_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) ->
↳ 'None'
```

```
batterySOH_EKF(PP_BatterySOH_EKF *config)
```

Block type BATTERY_SOH_EKF.

4.2.16 cepstrum

Real cepstrum: $c = \text{IFFT}(\log|X|)$.

```
cepstrum(fft_mag_instance: 'fftMagResolver') -> 'None'
```

```
cepstrum(fftMagResolver fftMagInstance)
```

Block type CEPSTRUM.

4.2.17 clarkeTransform

Clarke transform - converts balanced three-phase quantities (a,b,c) to orthogonal two-phase stationary frame (α, β).

```
clarkeTransform() -> 'None'
```

```
clarke(void)
```

In C++ the class is `clarke`.

Block type CLARKE.

4.2.18 comp

Op-amp style comparator with configurable saturation levels and differential inputs (4 inputs, no params).

```
comp() -> 'None'
```

```
comp()
```

Block type COMPARATOR.

4.2.19 compFilter

Linear complementary filter for sensor fusion:.

```
compFilter(config: 'PP_compFilter') -> 'None'
```

```
compFilter(PP_compFilter *config)
```

Block type COMP_FILTER.

4.2.20 complementaryAhrs

Simple complementary AHRS.

```
complementaryAhrs(alpha: 'float') -> 'None'
```

```
complementaryAhrs(float Alpha)
```

Block type COMPLEMENTARY_AHRS.

4.2.21 deadZone

Dead-zone nonlinearity that suppresses signals inside a tunable band (3 inputs, no params).

```
deadZone() -> 'None'
```

```
deadZone()
```

Block type DEADZONE.

4.2.22 decibelsConverter

Decibels converter - converts a linear signal amplitude to decibels.

```
decibelsConverter() -> 'None'
```

```
decibelsConverter()
```

Block type DECIBELS.

4.2.23 decisionTree

Tiny decision-tree inference.

```
decisionTree(K: 'int', C: 'int', feat_idx, thresh, left, right, leaf_val) -> 'None'
```

```
decisionTree(uint16_t K, uint16_t C, uint16_t NNodes, uint16_t* FeatIdx, float* Thresh,  
↳ uint16_t* Left, uint16_t* Right, float* LeafVal)
```

Block type DECISION_TREE.

4.2.24 demux

Demultiplexer node.

```
demux(n_outputs: 'int') -> 'None'
```

```
demux(uint8_t numOutputs)
```

Block type DEMUX.

4.2.25 dob

Disturbance Observer (DOB) for robust control.

```
dob(config: 'PP_DOBConfig') -> 'None'
```

```
dob(PP_DOBConfig *config)
```

Block type DOB.

4.2.26 eegBandpower

EEG bandpower from an upstream FFT_MAG block.

```
eegBandpower(fft_mag_instance: 'fftMagResolver') -> 'None'
```

```
eegBandpower(fftMagResolver fftMagInstance)
```

Block type EEG_BANDPOWER.

4.2.27 emgHudgins

EMG Hudgins time-domain feature set.

```
emgHudgins(window_N: 'int' = 256, zc_threshold: 'float' = 0.02) -> 'None'
```

```
emgHudgins(uint16_t WindowN, float ZcThreshold)
```

Block type EMG_HUDGINS.

4.2.28 encoder

Standalone quadrature encoder block for position and/or velocity sensing, independent from the motor block.

```
encoder(id_or_cfg: 'Union[PP_HwId, PP_encoderBlockCfg]', config: 'Optional[PP_encoderCfg]' =  
    ↪ None, outputs: 'Optional[Sequence[Union[PP_EncoderOutput, int]]]' = None, outputCount:  
    ↪ 'Optional[int]' = None) -> 'None'
```

```
encoder(PP_HwId id, const PP_encoderCfg *config, const PP_EncoderOutput *outputs, uint8_t  
    ↪ outputCount)
```

```
encoder(const PP_encoderBlockCfg *config)
```

Block type ENCODER.

4.2.29 esc

Extremum Seeking Control (ESC) for real-time optimization.

```
esc(config: 'PP_ESCConfig') -> 'None'
```

```
esc(PP_ESCConfig *config)
```

Block type ESC.

4.2.30 escPID

Extremum Seeking Control PID auto-tuner.

```
escPID(config: 'PP_ESC_PID_Config') -> 'None'
```

```
escPID(PP_ESC_PID_Config *config)
```

Block type ESC_PID.

4.2.31 expander

Expander node - replicates a single input signal to multiple outputs.

```
expander(nbrOfOutputs: 'int') -> 'None'
```

```
expander(uint8_t nbrOfOutputs)
```

Block type EXPANDER.

4.2.32 extSensor

External I/O module block: an ADC or DAC on SPI or I2C.

```
extSensor(hwId, partNumber, outScale: 'float' = 1.0) -> 'None'
```

```
extSensor(PP_HwId hwId, PP_SensorPartNumber partNumber, float outScale = 1.0f)
```

Block type EXT_IO_MODULE.

4.2.33 featureStack

Variable-K feature-vector concatenator.

```
featureStack(K: 'int') -> 'None'
```

```
featureStack(uint16_t K)
```

Block type FEATURE_STACK.

4.2.34 fuzzyPID

Fuzzy Logic PID controller.

```
fuzzyPID(config: 'PP_FuzzyPIDConfig') -> 'None'
```

```
fuzzyPID(PP_FuzzyPIDConfig *config)
```

Block type FUZZY_PID.

4.2.35 gain

Gain block - multiplies the input signal by a constant coefficient.

```
gain(gainValue: 'float') -> 'None'
```

```
gain(float gainValue)
```

Block type GAIN.

4.2.36 gainSchedulePID

Gain-Scheduled PID controller.

```
gainSchedulePID(config: 'PP_GainSchedulePIDConfig') -> 'None'
gainSchedulePID(PP_GainSchedulePIDConfig *config)
```

Block type GAIN_SCHEDULE_PID.

4.2.37 gainScheduler

Generic Gain Scheduler (lookup table with interpolation).

```
gainScheduler(config: 'PP_GainSchedulerConfig') -> 'None'
gainScheduler(PP_GainSchedulerConfig *config)
```

Block type GAIN_SCHEDULER.

4.2.38 gpsinsEKF

Extended Kalman Filter for GPS/INS fusion.

```
gpsinsEKF(config: 'PP_GPSINS_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
gpsinsEKF(PP_GPSINS_EKF *config)
```

Block type GPSINS_EKF.

4.2.39 gridSyncEKF

Extended Kalman Filter for grid synchronization.

```
gridSyncEKF(config: 'PP_GridSync_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
gridSyncEKF(PP_GridSync_EKF *config)
```

Block type GRID_SYNC_EKF.

4.2.40 gyroscope

```
gyroscope(outScale: 'float') -> 'None'
imu(PP_HwId id, float outScale)
```

In C++ the class is imu.

4.2.41 hjorth

Hjorth time-domain complexity stats.

```
hjorth(window_N: 'int' = 128) -> 'None'
hjorth(uint16_t WindowN)
```

Block type HJORTH.

4.2.42 hrvTime

HRV time-domain stats over RR-interval ring.

```
hrvTime(ring_N: 'int' = 64) -> 'None'
```

```
hrvTime(uint16_t RingN)
```

Block type HRV_TIME.

4.2.43 ilc

Iterative Learning Control (ILC) for improving tracking over repeated trials.

```
ilc(config: 'PP_ILCConfig') -> 'None'
```

```
ilc(PP_ILCConfig *config)
```

Block type ILC.

4.2.44 imu

Inertial Measurement Unit (IMU) interface block providing 3-axis accelerometer, gyroscope, or magnetometer measurements.

```
imu(id: 'PP_HwId', outScale: 'float') -> 'None'
```

```
imu(PP_HwId id, float outScale)
```

Block type IMU.

4.2.45 imuBiasEst

IMU GYROSCOPE bias estimator with variance-gated EMA - the accelerometer passes straight through.

```
imuBiasEst(var_window: 'int' = 64, rest_threshold: 'float' = 0.001, alpha: 'float' = 0.01) ->  
↪ 'None'
```

```
imuBiasEst(uint16_t VarWindow, float RestThreshold, float Alpha)
```

Block type IMU_BIAS_EST.

4.2.46 innovationChi2

Chi-square innovation gate for any Kalman / EKF / UKF residual stream.

```
innovationChi2(n_meas: 'int', threshold: 'float') -> 'None'
```

```
innovationChi2(uint16_t NMeas, float Threshold)
```

Block type INNOVATION_CHI2.

4.2.47 integratorICBlock

Integrator with an initial condition and an external reset.

```
integratorICBlock() -> 'None'
```

Block type INTEGRATOR_IC.

4.2.48 invClarkeTransform

Inverse Clarke transform - converts orthogonal stationary frame (α, β) back to three-phase (a,b).

```
invClarkeTransform() -> 'None'
```

```
invClarke(void)
```

In C++ the class is invClarke.

Block type INV_CLARKE.

4.2.49 invParkTransform

Inverse Park transform - rotates the rotating (d,q) frame back to the stationary (α, β) frame.

```
invParkTransform() -> 'None'
```

```
invPark(void)
```

In C++ the class is invPark.

Block type INV_PARK.

4.2.50 linearKF

Linear Kalman Filter (LKF) for optimal state estimation in linear systems corrupted by Gaussian noise.

```
linearKF(LKF: 'PP_linearKF', select: 'Optional[PP_Tune]' = None) -> 'None'
```

```
linearKF(PP_linearKF *LKF)
```

```
linearKF(PP_linearKF *LKF, PP_Tune select)
```

Block type LINEAR_KF.

4.2.51 live2PidAw

```
live2PidAw(mode: 'int' = 1, report: 'int' = 0) -> 'None'
```

```
live2PidAw(uint8_t mode, uint8_t report)
```

4.2.52 livePiAw

```
livePiAw(mode: 'int' = 1, report: 'int' = 0) -> 'None'
```

```
livePiAw(uint8_t mode, uint8_t report)
```

4.2.53 livePidAw

```
livePidAw(mode: 'int' = 1, report: 'int' = 0) -> 'None'
livePidAw(uint8_t mode, uint8_t report)
```

4.2.54 lms

Least Mean Squares (LMS) adaptive filter.

```
lms(config: 'PP_LMSConfig') -> 'None'
lms(PP_LMSConfig *config)
```

Block type LMS.

4.2.55 loadTorqueEKF

Extended Kalman Filter for load-torque estimation on a rotational mechanical system.

```
loadTorqueEKF(config: 'PP_LoadTorque_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) ->
↳ 'None'
loadTorqueEKF(PP_LoadTorque_EKF *config)
```

Block type LOAD_TORQUE_EKF.

4.2.56 loudness

Loudness computer node.

```
loudness() -> 'None'
loudness()
```

Block type LOUDNESS.

4.2.57 lqrServo

LQR (Linear-Quadratic Regulator) with servo (integral) action.

```
lqrServo(n: 'int', m: 'int', p: 'int', K, u_max: 'float') -> 'None'
lqrServo(uint16_t n, uint16_t m, uint16_t p, float* K, float UMax)
```

Block type LQR_SERVO.

4.2.58 madgwickAhRs

Madgwick gradient-descent AHRS.

```
madgwickAhRs(beta: 'float') -> 'None'
madgwickAhRs(float Beta)
```

Block type MADGWICK_AHRS.

4.2.59 mag2

2D Euclidean magnitude - computes the length of a 2D vector.

```
mag2() -> 'None'
```

```
mag2()
```

Block type MAG2.

4.2.60 mag3

3D Euclidean magnitude - computes the length of a 3D vector.

```
mag3() -> 'None'
```

```
mag3()
```

Block type MAG3.

4.2.61 magnetometer

```
magnetometer(outScale: 'float') -> 'None'
```

```
imu(PP_HwId id, float outScale)
```

In C++ the class is imu.

4.2.62 mahonyAhrs

Mahony complementary AHRS with gyro-offset estimation.

```
mahonyAhrs(Kp: 'float', Ki: 'float') -> 'None'
```

```
mahonyAhrs(float Kp, float Ki)
```

Block type MAHONY_AHRS.

4.2.63 merger

Merger node - combines multiple input signals into a single output by summation.

```
merger(nbrOfInputs: 'int') -> 'None'
```

```
merger(uint8_t nbrOfInputs)
```

Block type MERGER.

4.2.64 microphone

Captures a mono audio stream and outputs it as normalised floating point in the range -1.

```
microphone(hw_id: 'PP_HwId') -> 'None'
```

```
microphone(PP_HwId id)
```

Block type MICROPHONE.

4.2.65 mixerMultirotor

Multirotor mixer - thrust + roll/pitch/yaw torques -> per-motor PWM commands.

```
mixerMultirotor(frame_type: 'int' = 0, pwm_min: 'float' = 1000.0, pwm_max: 'float' = 2000.0)
↳ -> 'None'
```

```
mixerMultirotor(uint8_t FrameType, float PwmMin, float PwmMax)
```

Block type MIXER_MULTIROTOR.

4.2.66 modulo

Floored modulo - the remainder after division, taking the sign of the divisor.

```
modulo() -> 'None'
```

```
modulo(void)
```

Block type MODULO.

4.2.67 motor

DC motor hardware block with duty-cycle input and selectable measurement outputs.

```
motor(id_or_cfg: 'Union[PP_HwId, PP_motorBlockCfg]', config: 'Optional[PP_motorCfg]' = None,
↳ outputs: 'Optional[Sequence[Union[PP_MotorOutput, int]]]' = None, outputCount:
↳ 'Optional[int]' = None) -> 'None'
```

```
motor(PP_HwId id, const PP_motorCfg *config, const PP_MotorOutput *outputs, uint8_t
↳ outputCount)
motor(const PP_motorBlockCfg *config)
```

Block type MOTOR.

4.2.68 mrac

Model Reference Adaptive Control (MRAC).

```
mrac(config: 'PP_MRACConfig') -> 'None'
```

```
mrac(PP_MRACConfig *config)
```

Block type MRAC.

4.2.69 mracLyapunov

Model Reference Adaptive Control with Lyapunov stability guarantee.

```
mracLyapunov(config: 'PP_MRACLyapunovConfig') -> 'None'
```

```
mracLyapunov(PP_MRACLyapunovConfig *config)
```

Block type MRAC_LYAPUNOV.

4.2.70 mracPID

Model Reference Adaptive PID controller.

```
mracPID(config: 'PP_MRAC_PID_Config') -> 'None'
```

```
mracPID(PP_MRAC_PID_Config *config)
```

Block type MRAC_PID.

4.2.71 mux

Multiplexer node.

```
mux(n_inputs: 'int') -> 'None'
```

```
mux(uint8_t numInputs)
```

Block type MUX.

4.2.72 parkTransform

Park transform - rotates the stationary (α, β) frame to the rotor-aligned rotating (d,q) frame.

```
parkTransform() -> 'None'
```

```
park(void)
```

In C++ the class is park.

Block type PARK.

4.2.73 pcaProject

Offline-trained PCA projection: $y = W * (x - \mu)$.

```
pcaProject(K: 'int', P: 'int', W, mu) -> 'None'
```

```
pcaProject(uint16_t K, uint16_t P, float* W, float* mu)
```

Block type PCA_PROJECT.

4.2.74 perfMonitorPID

Performance-Monitored PID controller.

```
perfMonitorPID(config: 'PP_PerfMonPID_Config') -> 'None'
```

```
perfMonitorPID(PP_PerfMonPID_Config *config)
```

Block type PERF_MONITOR_PID.

4.2.75 pidAw

```
pidAw(ki: 'float', kp: 'float', kd: 'float', tau: 'float', u_min: 'float', u_max: 'float', Tt:
    ↳ 'float', mode: 'int' = 1, report: 'int' = 0) -> 'None'

pidAw(float ki, float kp, float kd, float tau, float u_min, float u_max, float Tt, uint8_t
    ↳ mode, uint8_t report)
```

4.2.76 pidFromModel

PID gains computed from an identified model.

```
pidFromModel(identifier: 'Union[RLS, RRLS]', lambda_c: 'float' = 0.1, gain_slew_per_s:
    ↳ 'float' = 2.0, kp_max: 'float' = 0.0, ki_max: 'float' = 0.0, kd_max: 'float' = 0.0,
    ↳ report_status: 'bool' = False) -> 'None'

pidFromModel(RLS identifier, float lambda_c, float gain_slew_per_s, float kp_max, float
    ↳ ki_max, float kd_max)
pidFromModel(RRLS identifier, float lambda_c, float gain_slew_per_s, float kp_max, float
    ↳ ki_max, float kd_max)
```

Block type PID_FROM_MODEL.

4.2.77 pmsmEKF

Extended Kalman Filter for sensorless PMSM rotor position and speed estimation.

```
pmsmEKF(config: 'PP_PMSM_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'

pmsmEKF(PP_PMSM_EKF *config)
```

Block type PMSM_EKF.

4.2.78 pwmIn

Measures an incoming pulse train on one of the five frequency inputs.

```
pwmIn(id_or_cfg: 'Union[PP_HwId, Sequence[PP_pwmInCfg]]', outScale: 'Optional[float]' = None,
    ↳ items: 'Optional[int]' = None) -> 'None'

pwmIn(PP_HwId id, float outScale)
pwmIn(PP_pwmInCfg *config, uint8_t items)
```

Block type PWM_IN.

4.2.79 pwmOut

PWM output block that drives hardware timer channels with configurable carrier frequency and duty-cycle saturation limits.

```
pwmOut(id_or_cfg: 'Union[PP_HwId, Sequence[PP_pwmOutCfg]]', frequency: 'Optional[float]' =
    ↳ None, dutyLowerLimit: 'Optional[float]' = None, dutyUpperLimit: 'Optional[float]' = None,
    ↳ items: 'Optional[int]' = None) -> 'None'
```

```
pwmOut(PP_HwId id, float frequency, float dutyLowerLimit, float dutyUpperLimit)
pwmOut(PP_pwmOutCfg *config, uint8_t items)
```

Block type PWM_OUT.

4.2.80 quantizer

Quantizer node that maps a continuous input to discrete levels.

```
quantizer() -> 'None'
quantizer(void)
```

Block type QUANTIZER.

4.2.81 quatFromEuler

Convert Euler angles (roll, pitch, yaw) to a quaternion using the ZYX intrinsic Tait-Bryan (aerospace) convention.

```
quatFromEuler() -> 'None'
quatFromEuler()
```

Block type QUAT_FROM_EULER.

4.2.82 quatMult

Hamilton product (multiplication) of two quaternions.

```
quatMult() -> 'None'
quatMult()
```

Block type QUAT_MULT.

4.2.83 quatNormConj

Quaternion normalise / conjugate / inverse selector.

```
quatNormConj() -> 'None'
quatNormConj()
```

Block type QUAT_NORM_CONJ.

4.2.84 quatToEuler

Convert a quaternion (w, x, y, z) to Euler angles (roll, pitch, yaw) using the ZYX intrinsic Tait-Bryan (aerospace) convention.

```
quatToEuler() -> 'None'
quatToEuler()
```

Block type QUAT_TO_EULER.

4.2.85 quatToRotmat

Convert a quaternion (w, x, y, z) to a 3×3 direction-cosine rotation matrix (row-major, 9 scalar outputs).

```
quatToRotmat() -> 'None'
```

```
quatToRotmat()
```

Block type QUAT_TO_ROTMAT.

4.2.86 rateLimiter

Slew-rate limiter - clamps the output rate of change to prevent excessively fast transitions.

```
rateLimiter() -> 'None'
```

```
rateLimiter(void)
```

Block type RATE_LIMITER.

4.2.87 recorder

Records its input into the board's permanent audio store.

```
recorder() -> 'None'
```

```
recorder()
```

Block type RECORDER.

4.2.88 relayAutoTune

Relay-based PID auto-tuning (Astrom-Hagglund method).

```
relayAutoTune(config: 'PP_RelayAutoTuneConfig') -> 'None'
```

```
relayAutoTune(PP_RelayAutoTuneConfig *config)
```

Block type RELAY_AUTOTUNE.

4.2.89 repetitiveCtrl

Repetitive Controller for rejecting periodic disturbances.

```
repetitiveCtrl(config: 'PP_RepCtrlConfig') -> 'None'
```

```
repetitiveCtrl(PP_RepCtrlConfig *config)
```

Block type REPETITIVE_CTRL.

4.2.90 replay

Plays an audio file that travels with the design.

```
replay() -> 'None'
```

```
replay()
```

Block type AUDIO_FILE.

4.2.91 ringMod

Ring modulator - multiplies two signals.

```
ringMod() -> 'None'
```

```
ringMod()
```

Block type RING_MOD.

4.2.92 robotLocEKF

Extended Kalman Filter for differential-drive robot localization.

```
robotLocEKF(config: 'PP_RobotLoc_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
```

```
robotLocEKF(PP_RobotLoc_EKF *config)
```

Block type ROBOT_LOC_EKF.

4.2.93 sampleHold

Sample-and-hold - latches the signal value on the rising edge of the trigger input.

```
sampleHold() -> 'None'
```

```
sampleHold(void)
```

Block type SAMPLE_HOLD.

4.2.94 sat

Saturation (clamp) nonlinearity with runtime-tunable limits (3 inputs, no params).

```
sat() -> 'None'
```

```
sat()
```

Block type SATURATION.

4.2.95 schmidt

Schmitt trigger - hysteresis comparator with two distinct thresholds to prevent output chatter.

```
schmidt() -> 'None'
```

```
schmidt()
```

Block type SCHMITT.

4.2.96 sign

Signum (sign) function extracting the sign of the input.

```
sign() -> 'None'
```

```
sign(void)
```

Block type SIGN.

4.2.97 sinTrackEKF

Extended Kalman Filter for sinusoidal signal tracking.

```
sinTrackEKF(config: 'PP_SinTrack_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
```

```
sinTrackEKF(PP_SinTrack_EKF *config)
```

Block type SIN_TRACK_EKF.

4.2.98 skewKurt

Welford 3rd/4th moments.

```
skewKurt(window_N: 'int' = 0) -> 'None'
```

```
skewKurt(uint16_t WindowN)
```

Block type SKEW_KURT.

4.2.99 smo

Sliding-mode observer for state estimation under model uncertainty.

```
smo(config: 'PP_SMO') -> 'None'
```

```
slidingModeObserver(PP_SMO *config)
```

In C++ the class is slidingModeObserver.

Block type SMO.

4.2.100 speaker

Plays a mono audio stream.

```
speaker(hw_id: 'PP_HwId') -> 'None'
```

```
speaker(PP_HwId id)
```

Block type SPEAKER.

4.2.101 spectralFeatures

Spectral features from FFT_MAG bins.

```
spectralFeatures(fft_mag_instance: 'fftMagResolver') -> 'None'  
spectralFeatures(fftMagResolver fftMagInstance)
```

Block type SPECTRAL_FEATURES.

4.2.102 stateSpace

Discrete-time linear state-space block.

```
stateSpace(config: 'PP_stateSpace') -> 'None'  
stateSpace(PP_stateSpace *config)
```

Block type STATE_SPACE.

4.2.103 stepAutoTune

Step-response-based PID auto-tuning.

```
stepAutoTune(config: 'PP_StepAutoTuneConfig') -> 'None'  
stepAutoTune(PP_StepAutoTuneConfig *config)
```

Block type STEP_AUTOTUNE.

4.2.104 stereoMixer

N-channel stereo mixer.

```
stereoMixer(NumChannels: 'int') -> 'None'  
stereoMixer(uint8_t NumChannels)
```

Block type STEREO_MIXER.

4.2.105 stereoPan

Constant-power stereo panner.

```
stereoPan() -> 'None'  
stereoPan()
```

Block type STEREO_PAN.

4.2.106 strPID

Self-Tuning Regulator PID (STR-PID).

```
strPID(config: 'PP_STR_PID_Config') -> 'None'  
strPID(PP_STR_PID_Config *config)
```

Block type STR_PID.

4.2.107 str_

Self-Tuning Regulator (STR) using online system identification and pole placement.

```
str_(config: 'PP_STRConfig') -> 'None'
```

```
str(PP_STRConfig *config)
```

In C++ the class is str.

Block type STR.

4.2.108 switchBlock

Conditional 2-to-1 signal selector based on a control signal.

```
switchBlock() -> 'None'
```

```
switchBlock(void)
```

Block type SWITCH.

4.2.109 thermalEKF

Extended Kalman Filter for thermal system estimation.

```
thermalEKF(config: 'PP_Thermal_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
```

```
thermalEKF(PP_Thermal_EKF *config)
```

Block type THERMAL_EKF.

4.2.110 tiltEKF

Extended Kalman Filter for single-axis tilt estimation.

```
tiltEKF(config: 'PP_Tilt_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
```

```
tiltEKF(PP_Tilt_EKF *config)
```

Block type TILT_EKF.

4.2.111 trajScurve

7-segment S-curve (jerk-limited) trajectory generator.

```
trajScurve() -> 'None'
```

```
trajScurve()
```

Block type TRAJ_SCURVE.

4.2.112 trajTrapezoid

Trapezoidal-velocity trajectory generator.

```
trajTrapezoid() -> 'None'
```

```
trajTrapezoid()
```

Block type TRAJ_TRAPEZOID.

4.2.113 wmrEKF

Extended Kalman Filter for Wheeled Mobile Robot (WMR) localization with magnetometer.

```
wmrEKF(config: 'PP_WMR_EKF', tune: 'PP_Tune' = <PP_Tune.PP_NoLiveTune: 1>) -> 'None'
```

```
wmrEKF(PP_WMR_EKF *config)
```

Block type WMR_EKF.

4.2.114 zScoreNorm

Welford running z-score normalizer.

```
zScoreNorm(train_window: 'int' = 256, epsilon: 'float' = 1e-09) -> 'None'
```

```
zScoreNorm(uint16_t TrainWindowN, float Epsilon)
```

Block type ZSCORE_NORM.

4.3 Macros (15)

4.3.1 ParamEQBand

TRUE RBJ band (peaking) parametric equalizer - boosts or cuts a band while leaving the rest of the spectrum at unity gain.

```
ParamEQBand(Fc1: 'float', Fc2: 'float', FilterOrder: 'int', Gain: 'float') -> 'None'
```

```
ParamEQBand(float Fc1, float Fc2, uint8_t FilterOrder, float Gain)
```

Block type PARAM_EQ_BAND.

4.3.2 ParamEQHigh

TRUE RBJ high-shelf parametric equalizer - boosts or cuts frequencies above the corner frequency while leaving lower frequencies at unity gain.

```
ParamEQHigh(Fc: 'float', FilterOrder: 'int', Gain: 'float') -> 'None'
```

```
ParamEQHigh(float Fc, uint8_t FilterOrder, float Gain)
```

Block type PARAM_EQ_HIGH.

4.3.3 ParamEQPeak

TRUE RBJ peaking parametric equalizer - boosts or cuts a narrow band centered on a specific frequency.

```
ParamEQPeak(Fccenter: 'float', Gain: 'float') -> 'None'
```

```
ParamEQPeak(float Fc, float Gain)
```

Block type PARAM_EQ_PEAK.

4.3.4 butterBq

Butterworth filter implemented as a cascade of second-order sections (biquads) for improved numerical stability.

```
butterBq(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType'  
↪ = <PP_FilterType.NoFilterType: 255>) -> 'None'
```

```
butterBq(uint8_t n, float fc, PP_FilterType ftype)
```

```
butterBq(uint8_t n, float fc1, float fc2, PP_FilterType ftype)
```

Block type BUTTERWORTH_BIQUAD.

4.3.5 cheby1Bq

Chebyshev Type I IIR filter implemented as a cascade of biquads (second-order sections).

```
cheby1Bq(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType'  
↪ = <PP_FilterType.NoFilterType: 255>, chebyCfg: 'int' = 255) -> 'None'
```

```
cheby1Bq(uint8_t n, float fc, PP_FilterType ftype, PP_Cheby1Cfg chebyCfg)
```

```
cheby1Bq(uint8_t n, float fc1, float fc2, PP_FilterType ftype, PP_Cheby1Cfg chebyCfg)
```

Block type CHEBYSHEV1_BIQUAD.

4.3.6 cheby2Bq

Chebyshev Type II (inverse Chebyshev) IIR filter implemented as a cascade of biquads (second-order sections).

```
cheby2Bq(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType'  
↪ = <PP_FilterType.NoFilterType: 255>, chebyCfg: 'int' = 255) -> 'None'
```

```
cheby2Bq(uint8_t n, float fc, PP_FilterType ftype, PP_Cheby2Cfg chebyCfg)
```

```
cheby2Bq(uint8_t n, float fc1, float fc2, PP_FilterType ftype, PP_Cheby2Cfg chebyCfg)
```

Block type CHEBYSHEV2_BIQUAD.

4.3.7 decimation

Decimation block - decreases the sampling rate by an integer factor with anti-aliasing filtering.

```
decimation(Buffer: 'Sequence[float]') -> 'None'
```

```
decimation(float *Buffer)
decimation(uint8_t factor, float gain, float fc, uint8_t filter_order)
```

Block type DECIMATION.

4.3.8 ellipBq

Elliptic (Cauer) IIR filter implemented as a cascade of biquads (second-order sections).

```
ellipBq(n: 'int', fc_or_fc1: 'float', fc2: 'Optional[float]' = None, ftype: 'PP_FilterType' =
↳ <PP_FilterType.NoFilterType: 255>, ellipCfg: 'int' = 255) -> 'None'

ellipBq(uint8_t n, float fc, PP_FilterType ftype, PP_EllipCfg ellipCfg)
ellipBq(uint8_t n, float fc1, float fc2, PP_FilterType ftype, PP_EllipCfg ellipCfg)
```

Block type ELLIPTIC_BIQUAD.

4.3.9 live2Pid

2-DOF Live PID controller - Two Degrees-of-Freedom PID with separate setpoint weighting for reference tracking and disturbance rejection.

```
live2Pid() -> 'None'
live2Pid()
```

Block type LIVE_2PID.

4.3.10 livePd

Live PD controller - Proportional-Derivative controller with gains adjustable in real time via input ports.

```
livePd() -> 'None'
livePd()
```

Block type LIVE_PD.

4.3.11 livePi

Live PI controller - Proportional-Integral controller with gains adjustable in real time via input ports.

```
livePi() -> 'None'
livePi()
```

Block type LIVE_PI.

4.3.12 livePid

Live PID controller - Proportional-Integral-Derivative controller with gains adjustable in real time via input ports.

```
livePid() -> 'None'
```

```
livePid()
```

Block type LIVE_PID.

4.3.13 paramEQLow

TRUE RBJ low-shelf parametric equalizer - boosts or cuts frequencies below the corner frequency while leaving higher frequencies at unity gain.

```
paramEQLow(Fc: 'float', FilterOrder: 'int', Gain: 'float') -> 'None'
```

```
paramEQLow(float Fc, uint8_t FilterOrder, float Gain)
```

Block type PARAM_EQ_LOW.

4.3.14 rls

```
rls(modelOrd: 'int', gain: 'float', system: 'macro') -> 'None'
```

```
rls(uint8_t modelOrd, float gain, macro system)
```

4.3.15 sineAm

Measures the amplitude of a sine component already present in the input, at a frequency you specify.

```
sineAm(sineFreq: 'float') -> 'None'
```

```
sineAm(float sineFreq)
```

Block type SINE_AM.

Chapter 5

The objects a program holds

A deployed design is reached through three kinds of object: the probes you read, the inputs you write, and the blocks themselves. The board object opens the link and moves data; these carry the values.

5.1 probe

A reading taken out of the running design. One probe is one scalar, sampled at the design's rate.

5.1.1 probe (constructor)

[PPL_API_1105]:

Service Name	probe (constructor)
Syntax (Python)	probe(label: Optional[str] = None) -> None
Parameters (in)	- label (Optional[str], default None)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Create the object and add it to the design being described. It must be created between startDesign and stopDesign.
Available via	papaya (Python)

5.1.2 probe.getVal

[PPL_API_1110]:

Service Name	probe.getVal
Syntax (Python)	getVal(id: Optional[int] = None) -> float
Parameters (in)	- id (Optional[int], default None)
Parameters (inout)	None
Parameters (out)	None
Return value	float.
Description	The probe's most recent value. Call getProbes on the board object first, or you will read the value from the previous refresh.
Available via	papaya (Python)

5.1.3 probe.getId

[PPL_API_1115]:

Service Name	probe.getId
Syntax (Python)	getId() -> int
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	int.
Description	The index the board knows this probe by. Needed only when reading a raw probe frame yourself.
Available via	papaya (Python)

5.1.4 probe.show

[PPL_API_1120]:

Service Name	probe.show
Syntax (Python)	show() -> None
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None

Service Name	probe.show
Description	Mark the probe visible in the Monitor. A probe hidden here still streams; it is simply not drawn.
Available via	papaya (Python)

5.2 input

A value the host writes into the running design. The only way to change a design's behaviour while it runs without redeploying it.

5.2.1 input (constructor)

[PPL_API_1125]:

Service Name	input (constructor)
Syntax (Python)	<code>input(label: Optional[str] = None, val: Optional[float] = None) -> None</code>
Parameters (in)	- label (Optional[str], default None) - val (Optional[float], default None)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Create the object and add it to the design being described. It must be created between startDesign and stopDesign.
Available via	papaya (Python)

5.2.2 input.enableRemote

[PPL_API_1130]:

Service Name	input.enableRemote
Syntax (Python)	<code>enableRemote() -> None</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None

Service Name	input.enableRemote
Description	Allow this input to be written by name from another program or from the Monitor. Call it while the design is being described, before stopDesign.
Available via	papaya (Python)

5.2.3 input.addToQueue

[PPL_API_1135]:

Service Name	input.addToQueue
Syntax (Python)	addToQueue(input_value: float, id: Optional[int] = None) -> None
Parameters (in)	- input_value (float) - id (Optional[int], default None)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Stage a value. Nothing reaches the board until setInputs sends the whole queue, so several inputs change in the same period rather than one at a time.
Available via	papaya (Python)

5.2.4 input.getId

[PPL_API_1140]:

Service Name	input.getId
Syntax (Python)	getId() -> int
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	int.
Description	The index the board knows this input by.
Available via	papaya (Python)

5.3 VectorProbe

A reading that is a whole array rather than one number: a spectrum, a filter's impulse response, a buffered frame.

5.3.1 VectorProbe (constructor)

[PPL_API_1145]:

Service Name	VectorProbe (constructor)
Syntax (Python)	VectorProbe(label: Optional[str] = None) -> None
Parameters (in)	- label (Optional[str], default None)
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Create the object and add it to the design being described. It must be created between startDesign and stopDesign.
Available via	papaya (Python)

5.3.2 VectorProbe.getSamples

[PPL_API_1150]:

Service Name	VectorProbe.getSamples
Syntax (Python)	getSamples(samples_size: int) -> bytes
Parameters (in)	- samples_size (int)
Parameters (inout)	None
Parameters (out)	None
Return value	bytes.
Description	Read one frame. samples_size is the number of values to take, which must not exceed the length the block declares.
Available via	papaya (Python)

5.3.3 VectorProbe.addToQueue

[PPL_API_1155]:

Service Name	VectorProbe.addToQueue
Syntax (Python)	addToQueue() -> None
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Stage a frame to be written back into the design.
Available via	papaya (Python)

5.3.4 VectorProbe.show

[PPL_API_1160]:

Service Name	VectorProbe.show
Syntax (Python)	show() -> None
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Mark the vector probe visible in the Monitor.
Available via	papaya (Python)

5.3.5 VectorProbe.getId

[PPL_API_1165]:

Service Name	VectorProbe.getId
Syntax (Python)	getId() -> int
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	int.
Description	The index the board knows this vector probe by.
Available via	papaya (Python)

Chapter 6

Sources and outputs

Two classes carry a family of factory methods, one per waveform. They are what the examples use, and they read better than the constructor because the shape is in the name rather than in a mode argument.

6.1 The arguments every factory accepts

Beyond the arguments that decide the shape, all of them take the same five keyword arguments. They are listed here once.

Argument	Meaning
amplitude	Peak value of the generated shape. Default 1.0.
offset	Added to every sample after amplitude is applied. Default 0.0.
mode	A PP_WaveMode: run forever, run n_periods times, run once, or wait for a trigger.
n_periods	How many periods PP_WaveMode.RepeatN runs for. Ignored in the other modes.
trig_pol	A PP_WaveTrigPol: which edge or level starts a triggered waveform.

6.2 signalSource

Generate a waveform inside the design, with no wire and no hardware. The factories below are the ordinary way to make one; the bare constructor exists for a shape none of them covers.

Factory	Shape it makes
<code>signalSource.arbitrary(samples, freq=None, phase=0.0)</code>	Play the lookup table <code>samples</code> , resampled so one pass takes $1 / \text{freq}$ seconds.
<code>signalSource.chirp_linear(f0, f1, t_sweep, phase=0.0)</code>	A sweep from <code>f0</code> to <code>f1</code> hertz over <code>t_sweep</code> seconds, the frequency rising by equal steps in hertz.
<code>signalSource.chirp_log(f0, f1, t_sweep, phase=0.0)</code>	A sweep from <code>f0</code> to <code>f1</code> hertz over <code>t_sweep</code> seconds, the frequency rising by equal ratios, so it spends the same time in each octave.
<code>signalSource.dataset(samples, phase=0.0)</code>	Play a recording. Hand it the <code>samples</code> and nothing else; the upload happens when the design is deployed.
<code>signalSource.dc(value=0.0)</code>	A constant value. Useful as a setpoint you later drive from an input.
<code>signalSource.gauspuls(freq, bw_fraction=0.5, prf=10.0)</code>	A Gaussian-enveloped burst at <code>freq</code> hertz whose bandwidth is <code>bw_fraction</code> of the carrier, repeating <code>prf</code> times a second.
<code>signalSource.impulse(t_impulse=0.0)</code>	One sample of full amplitude at <code>t_impulse</code> seconds, zero elsewhere.
<code>signalSource.multisine(f0, df, n_lines=8, phase_scheme=PP_WaveMultisinePhase.Schroeder, lfsr_seed=44257)</code>	<code>n_lines</code> sinusoids starting at <code>f0</code> and spaced <code>df</code> apart, with their phases chosen by <code>phase_scheme</code> so the peaks do not pile up. The usual excitation for identification.
<code>signalSource.prbs(bits=9, clock_div=1, lfsr_seed=44257)</code>	A pseudo-random binary sequence of <code>bits</code> stages, changing state every <code>clock_div</code> samples. Repeats after 2 to the power <code>bits</code> minus 1 steps.
<code>signalSource.pulse(freq, duty, phase=0.0)</code>	A rectangular wave at <code>freq</code> hertz, high for the fraction <code>duty</code> of each period. <code>duty</code> is a fraction, so 0.25 is a quarter.
<code>signalSource.pulse_train(freq, duty, n_pulses, prf, phase=0.0)</code>	<code>n_pulses</code> rectangular pulses at <code>freq</code> hertz, the burst repeating <code>prf</code> times a second.
<code>signalSource.ramp(slope, t_start=0.0, sat_low=-1000000000.0, sat_high=1000000000.0)</code>	A straight line of gradient <code>slope</code> per second, starting at <code>t_start</code> , held between <code>sat_low</code> and <code>sat_high</code> .
<code>signalSource.sawtooth(freq, phase=0.0, direction=0)</code>	A sawtooth at <code>freq</code> hertz. <code>direction</code> 0 rises, 1 falls.
<code>signalSource.sinc(freq, n_lobes=5, phase=0.0)</code>	A sinc centred on the middle of its window, <code>n_lobes</code> either side, whose first null is at <code>freq</code> .
<code>signalSource.sine(freq, phase=0.0)</code>	A sine at <code>freq</code> hertz.

Factory	Shape it makes
<code>signalSource.square(freq, phase=0.0)</code>	A square wave at freq hertz, half the period high.
<code>signalSource.step(t_step, value_before=0.0, value_after=1.0)</code>	value_before until t_step seconds, value_after after it.
<code>signalSource.triangle(freq, phase=0.0, symmetry=0.5)</code>	A triangle at freq hertz. symmetry moves the peak: 0.5 is symmetric, 0 and 1 are the two sawtooths.

6.3 analogOut

Drive a physical analog output. The factories mirror `signalSource` exactly, with a hardware channel as the first argument, so a waveform can be moved from a probe to a pin by changing one call.

Factory	Shape it makes
<code>analogOut.chirpLinear(hw_id, f0, f1, t_sweep, phase=0.0)</code>	
<code>analogOut.chirpLog(hw_id, f0, f1, t_sweep, phase=0.0)</code>	
<code>analogOut.dc(hw_id)</code>	A constant value. Useful as a setpoint you later drive from an input.
<code>analogOut.gauspuls(hw_id, freq, bw_fraction=0.5, prf=10.0)</code>	A Gaussian-enveloped burst at freq hertz whose bandwidth is bw_fraction of the carrier, repeating prf times a second.
<code>analogOut.impulse(hw_id, t_impulse=0.0)</code>	One sample of full amplitude at t_impulse seconds, zero elsewhere.
<code>analogOut.multisine(hw_id, f0, df, n_lines=8, phase_scheme=0, lfsr_seed=44257)</code>	n_lines sinusoids starting at f0 and spaced df apart, with their phases chosen by phase_scheme so the peaks do not pile up. The usual excitation for identification.
<code>analogOut.prbs(hw_id, bits=9, clock_div=1, lfsr_seed=44257)</code>	A pseudo-random binary sequence of bits stages, changing state every clock_div samples. Repeats after 2 to the power bits minus 1 steps.
<code>analogOut.pulse(hw_id, freq, phase=0.0, duty=0.5)</code>	A rectangular wave at freq hertz, high for the fraction duty of each period. duty is a fraction, so 0.25 is a quarter.
<code>analogOut.pulseTrain(hw_id, freq, phase=0.0, duty=0.5, n_pulses=1, prf=10.0)</code>	

Factory	Shape it makes
<code>analogOut.ramp(hw_id, slope=1.0, t_start=0.0, sat_low=-10.0, sat_high=10.0)</code>	A straight line of gradient slope per second, starting at <code>t_start</code> , held between <code>sat_low</code> and <code>sat_high</code> .
<code>analogOut.sawtooth(hw_id, freq, phase=0.0, direction=0)</code>	A sawtooth at <code>freq</code> hertz. direction 0 rises, 1 falls.
<code>analogOut.sinc(hw_id, freq, phase=0.0, n_lobes=5)</code>	A sinc centred on the middle of its window, <code>n_lobes</code> either side, whose first null is at <code>freq</code> .
<code>analogOut.sine(hw_id, freq, phase=0.0)</code>	A sine at <code>freq</code> hertz.
<code>analogOut.square(hw_id, freq, phase=0.0)</code>	A square wave at <code>freq</code> hertz, half the period high.
<code>analogOut.step(hw_id, t_step=0.0, value_before=0.0, value_after=1.0)</code>	<code>value_before</code> until <code>t_step</code> seconds, <code>value_after</code> after it.
<code>analogOut.triangle(hw_id, freq, phase=0.0, symmetry=0.5)</code>	A triangle at <code>freq</code> hertz. <code>symmetry</code> moves the peak: 0.5 is symmetric, 0 and 1 are the two sawtooths.

Chapter 7

Opening the link from a host program

Every generated PC program starts and ends with the two calls below, and a program that wants to work with a design already on the board starts with the third. The contract used to be described in the *PAPAYA Studio User Manual*, which is the wrong volume: the manual teaches the editor that writes the call, and the call itself is reference material.

7.0.1 connect_usb

[PPL_API_1505]:

Service Name	connect_usb
Syntax (Python)	connect_usb() -> bool
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	bool - TRUE if the call succeeded, FALSE if it did not.
Description	Open the USB link to the board and make it the link every later call uses. Call it once, before any board object is used. Calling it a second time while a link is open is refused rather than silently opening a second one.
Available via	papaya (Python)

7.0.2 disconnect_usb

[PPL_API_1510]:

Service Name	disconnect_usb
Syntax (Python)	<code>disconnect_usb()</code> -> None
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	None
Description	Close the link and release the port. A program that exits without calling it leaves the port claimed until the process ends.
Available via	papaya (Python)

7.0.3 attach

[PPL_API_1515]:

Service Name	attach
Syntax (Python)	<code>attach(board_id: int = 0, *, connect: bool = True, source: str = auto, design: Optional[str] = None)</code> -> RemoteDesign
Parameters (in)	- board_id (int, default 0) - connect (bool, default True) - source (str, default auto) - design (Optional[str], default None)
Parameters (inout)	None
Parameters (out)	None
Return value	RemoteDesign.
Description	Connect to a board and read back whatever design it is already carrying, so a program can drive a design it did not deploy. Use it as a context manager and the link closes for you.
Available via	papaya (Python)

Chapter 8

Types

The enumerations and configuration structures a caller passes in. Members are always written qualified, `PP_C2dAlgo.Tustin` rather than `Tustin`.

8.1 Enumerations

8.1.1 PP_AnOutMode

[PPL_API_1705]:

Name	PP_AnOutMode
Kind	enumeration
Elements	- <code>PP_AnOutMode.InputDriven = 0</code> - <code>PP_AnOutMode.PredefinedWaveform = 1</code> - <code>PP_AnOutMode.NoAnOutMode = 255</code> (unset)
Description	Whether an analog output follows a wire in the design or plays a waveform generated on the board.
Available via	papaya (Python)

8.1.2 PP_C2dAlgo

[PPL_API_1710]:

Name	PP_C2dAlgo
Kind	enumeration

Name	PP_C2dAlgo
Elements	- PP_C2dAlgo.Tustin = 0 - PP_C2dAlgo.EulerBackward = 1 - PP_C2dAlgo.EulerForward = 2 - PP_C2dAlgo.NoC2dAlgo = 255 (unset)
Description	Which discretisation rule turns a continuous design into the coefficients the board runs. Chosen once per design, at startDesign.
Available via	papaya (Python)

8.1.3 PP_Cheby1Cfg

[PPL_API_1715]:

Name	PP_Cheby1Cfg
Kind	enumeration
Elements	- PP_Cheby1Cfg.Cheby1Cfg1 = 0 - PP_Cheby1Cfg.Cheby1Cfg2 = 1 - PP_Cheby1Cfg.Cheby1Cfg3 = 2 - PP_Cheby1Cfg.Cheby1Cfg4 = 3 - PP_Cheby1Cfg.Cheby1Cfg5 = 4 - PP_Cheby1Cfg.NoCheby1Cfg = 255 (unset)
Description	Choices accepted where a PP_Cheby1Cfg is expected.
Available via	papaya (Python)

8.1.4 PP_Cheby2Cfg

[PPL_API_1720]:

Name	PP_Cheby2Cfg
Kind	enumeration
Elements	- PP_Cheby2Cfg.Cheby2Cfg1 = 0 - PP_Cheby2Cfg.Cheby2Cfg2 = 1 - PP_Cheby2Cfg.Cheby2Cfg3 = 2 - PP_Cheby2Cfg.Cheby2Cfg4 = 3 - PP_Cheby2Cfg.Cheby2Cfg5 = 4 - PP_Cheby2Cfg.NoCheby2Cfg = 255 (unset)
Description	Choices accepted where a PP_Cheby2Cfg is expected.
Available via	papaya (Python)

8.1.5 PP_ComStatus

[PPL_API_1725]:

Name	PP_ComStatus
Kind	enumeration
Elements	- PP_ComStatus.ComOk = 0 - PP_ComStatus.WrongSeqNumber = 1 - PP_ComStatus.WrongCrc = 2 - PP_ComStatus.UnknownComError = 3 - PP_ComStatus.NoComStatus = 255 (unset)
Description	The state of the link itself, independent of the design.
Available via	papaya (Python)

8.1.6 PP_DesignStatus

[PPL_API_1730]:

Name	PP_DesignStatus
Kind	enumeration
Elements	The 23 members are listed below the table.
Description	What the board thinks of the design it was sent. DesignOk is the only value that means the design is running; every other value names the reason it is not. BadRequest is the value 8 that a refused deployment reports.
Available via	papaya (Python)

- PP_DesignStatus.DesignOk = 0
- PP_DesignStatus.MemError = 1
- PP_DesignStatus.TfError = 2
- PP_DesignStatus.UnknownC2dAlgo = 3
- PP_DesignStatus.TextOk = 4
- PP_DesignStatus.TxtError = 5
- PP_DesignStatus.BadParam = 6
- PP_DesignStatus.BadConnection = 7
- PP_DesignStatus.BadRequest = 8
- PP_DesignStatus.SamplingFreqOutOfLimit = 9
- PP_DesignStatus.GPIO_Failure = 10
- PP_DesignStatus.TIM_Failure = 11
- PP_DesignStatus.ADC_Failure = 12
- PP_DesignStatus.DAC_Failure = 13
- PP_DesignStatus.COMP_Failure = 14

- `PP_DesignStatus.DMA_Failure = 15`
- `PP_DesignStatus.OPAMP_Failure = 16`
- `PP_DesignStatus.SPI_Failure = 17`
- `PP_DesignStatus.UART_Failure = 18`
- `PP_DesignStatus.I2C_Failure = 19`
- `PP_DesignStatus.USB_Failure = 20`
- `PP_DesignStatus.UndefinedFailure = 21`
- `PP_DesignStatus.NoDesignStatus = 255 (unset)`

8.1.7 PP_EllipCfg

[PPL_API_1735]:

Name	PP_EllipCfg
Kind	enumeration
Elements	- <code>PP_EllipCfg.EllipCfg1 = 0</code> - <code>PP_EllipCfg.EllipCfg2 = 1</code> - <code>PP_EllipCfg.EllipCfg3 = 2</code> - <code>PP_EllipCfg.EllipCfg4 = 3</code> - <code>PP_EllipCfg.NoEllipCfg = 255 (unset)</code>
Description	Choices accepted where a <code>PP_EllipCfg</code> is expected.
Available via	papaya (Python)

8.1.8 PP_EncoderOutput

[PPL_API_1740]:

Name	PP_EncoderOutput
Kind	enumeration
Elements	- <code>PP_EncoderOutput.EncoderVelocity = 0</code> - <code>PP_EncoderOutput.EncoderPosition = 1</code> - <code>PP_EncoderOutput.NoEncoderOutput = 255 (unset)</code>
Description	Which quantity an encoder block reports.
Available via	papaya (Python)

8.1.9 PP_FilterType

[PPL_API_1745]:

Name	PP_FilterType
Kind	enumeration
Elements	- PP_FilterType.passLow = 0 - PP_FilterType.passHigh = 1 - PP_FilterType.passBand = 2 - PP_FilterType.stopBand = 3 - PP_FilterType.NoFilterType = 255 (unset)
Description	Which band a filter passes.
Available via	papaya (Python)

8.1.10 PP_FirWindowType**[PPL_API_1750]:**

Name	PP_FirWindowType
Kind	enumeration
Elements	The 10 members are listed below the table.
Description	The window applied to a windowed-sinc filter design. It trades transition width against stopband depth.
Available via	papaya (Python)

- PP_FirWindowType.hamming = 0
- PP_FirWindowType.blackMan = 1
- PP_FirWindowType.rectangular = 2
- PP_FirWindowType.hann = 3
- PP_FirWindowType.bartlett = 4
- PP_FirWindowType.flatTop = 5
- PP_FirWindowType.blackManHarris = 6
- PP_FirWindowType.kaiser5 = 7
- PP_FirWindowType.kaiser8 = 8
- PP_FirWindowType.NoFirWindowType = 255 (unset)

8.1.11 PP_Hwld**[PPL_API_1755]:**

Name	PP_Hwld
Kind	enumeration
Elements	The 60 members are listed below the table.

Name	PP_HwId
Description	The board's physical channels, named the way the Studio names them. See the <i>PAPAYA Board Datasheet</i> for what each one is wired to and what it can measure.
Available via	papaya (Python)

- `PP_HwId.MOTOR1 = 0`
- `PP_HwId.MOTOR2 = 1`
- `PP_HwId.FI1_PULSE = 256`
- `PP_HwId.FI2_PULSE = 257`
- `PP_HwId.FI3_PULSE = 258`
- `PP_HwId.FI4_PULSE = 259`
- `PP_HwId.FI5_PULSE = 260`
- `PP_HwId.FI1_PERIOD = 512`
- `PP_HwId.FI2_PERIOD = 513`
- `PP_HwId.FI3_PERIOD = 514`
- `PP_HwId.FI4_PERIOD = 515`
- `PP_HwId.FI5_PERIOD = 516`
- `PP_HwId.FI1_DELAY = 768`
- `PP_HwId.FI2_DELAY = 769`
- `PP_HwId.FI3_DELAY = 770`
- `PP_HwId.FI4_DELAY = 771`
- `PP_HwId.FI5_DELAY = 772`
- `PP_HwId.FO1 = 1024`
- `PP_HwId.FO2 = 1025`
- `PP_HwId.FO3 = 1026`
- `PP_HwId.FO4 = 1027`
- `PP_HwId.FO5 = 1028`
- `PP_HwId.FO6 = 1029`
- `PP_HwId.FO7 = 1030`
- `PP_HwId.FO8 = 1031`
- `PP_HwId.AO1 = 1280`
- `PP_HwId.AO2 = 1281`
- `PP_HwId.AO3 = 1282`
- `PP_HwId.AO4 = 1283`
- `PP_HwId.AO5 = 1284`
- `PP_HwId.AIN1 = 1536`
- `PP_HwId.AIN2 = 1537`
- `PP_HwId.AIN3 = 1538`
- `PP_HwId.AIN4 = 1539`
- `PP_HwId.AIN5 = 1540`
- `PP_HwId.AIN6 = 1541`
- `PP_HwId.AIN7 = 1542`
- `PP_HwId.AIN8 = 1543`

- `PP_HwId.AIN9` = 1544
- `PP_HwId.AIN10` = 1545
- `PP_HwId.MIC_L` = 1792
- `PP_HwId.MIC_R` = 1793
- `PP_HwId.SPEAKER1` = 2048
- `PP_HwId.SPEAKER2` = 2049
- `PP_HwId.RECORDER_1` = 3072
- `PP_HwId.REPLAY_1` = 3328
- `PP_HwId.IMU_ACC` = 2304
- `PP_HwId.IMU_GYR` = 2305
- `PP_HwId.IMU_MAG` = 2306
- `PP_HwId.SPI1` = 2560
- `PP_HwId.SPI2` = 2561
- `PP_HwId.SPI3` = 2562
- `PP_HwId.SPI4` = 2563
- `PP_HwId.I2C1` = 2564
- `PP_HwId.I2C2` = 2565
- `PP_HwId.I2C3` = 2566
- `PP_HwId.I2C4` = 2567
- `PP_HwId.ENC1` = 2816
- `PP_HwId.ENC2` = 2817
- `PP_HwId.NoHwRoutine` = 65535 (unset)

8.1.12 PP_MonInterface

[PPL_API_1760]:

Name	PP_MonInterface
Kind	enumeration
Elements	- <code>PP_MonInterface.MonitoringOverUsb</code> = 0 - <code>PP_MonInterface.MonitoringOverSpi</code> = 1 - <code>PP_MonInterface.NoMonInterface</code> = 255 (unset)
Description	Which physical link carries monitoring traffic.
Available via	papaya (Python)

8.1.13 PP_MonitorMode

[PPL_API_1765]:

Name	PP_MonitorMode
Kind	enumeration

Name	PP_MonitorMode
Elements	- PP_MonitorMode.Live = 0 - PP_MonitorMode.Offline = 1 - PP_MonitorMode.Dual = 2 - PP_MonitorMode.NoMonitorMode = 255 (unset)
Description	Whether samples stream out as they are taken, are kept on the board for later download, or both.
Available via	papaya (Python)

8.1.14 PP_MotorOutput

[PPL_API_1770]:

Name	PP_MotorOutput
Kind	enumeration
Elements	- PP_MotorOutput.MotorVelocity = 0 - PP_MotorOutput.MotorPosition = 1 - PP_MotorOutput.MotorCurrent = 2 - PP_MotorOutput.NoMotorOutput = 255 (unset)
Description	Which quantity a motor block reports.
Available via	papaya (Python)

8.1.15 PP_NodeType

[PPL_API_1775]:

Name	PP_NodeType
Kind	enumeration
Elements	- PP_NodeType.Sum = 0 - PP_NodeType.Mul = 1 - PP_NodeType.NoNodeType = 255 (unset)
Description	Whether a node adds its inputs or multiplies them.
Available via	papaya (Python)

8.1.16 PP_NoiseType

[PPL_API_1780]:

Name	PP_NoiseType
Kind	enumeration
Elements	- PP_NoiseType.uniform = 0 - PP_NoiseType.gaussian = 1
Description	The distribution a noise source draws from.
Available via	papaya (Python)

8.1.17 PP_SensorPartNumber**[PPL_API_1785]:**

Name	PP_SensorPartNumber
Kind	enumeration
Elements	The 53 members are listed below the table.
Description	The sensors a design can read directly. These are parts you connect to the board's serial buses, not anything fitted to it: choosing one here tells the block which register map and scale factors to use, so you do not have to write a driver.
Available via	papaya (Python)

- PP_SensorPartNumber.SENSOR_NONE = 0
- PP_SensorPartNumber.LIS3DH = 1
- PP_SensorPartNumber.ADXL345 = 2
- PP_SensorPartNumber.ADXL362 = 3
- PP_SensorPartNumber.BMA400 = 4
- PP_SensorPartNumber.LIS2DW12 = 5
- PP_SensorPartNumber.MPU6050 = 6
- PP_SensorPartNumber.LSM6DSO = 7
- PP_SensorPartNumber.BMI270 = 8
- PP_SensorPartNumber.ICM42688P = 9
- PP_SensorPartNumber.ISM330DHCX = 10
- PP_SensorPartNumber.LSM6DSV16X = 11
- PP_SensorPartNumber.MPU9250 = 12
- PP_SensorPartNumber.ICM20948 = 13
- PP_SensorPartNumber.LSM9DS1 = 14
- PP_SensorPartNumber.BMM150 = 15
- PP_SensorPartNumber.MMC5983MA = 16
- PP_SensorPartNumber.LIS3MDL = 17
- PP_SensorPartNumber.BME280 = 18

- `PP_SensorPartNumber.BMP280 = 19`
- `PP_SensorPartNumber.BME680 = 20`
- `PP_SensorPartNumber.BMP390 = 21`
- `PP_SensorPartNumber.LPS22HH = 22`
- `PP_SensorPartNumber.DPS310 = 23`
- `PP_SensorPartNumber.MS5611 = 24`
- `PP_SensorPartNumber.HTS221 = 25`
- `PP_SensorPartNumber.MAX86141 = 26`
- `PP_SensorPartNumber.MAX30001 = 27`
- `PP_SensorPartNumber.ADS1292R = 28`
- `PP_SensorPartNumber.ADS1298R = 29`
- `PP_SensorPartNumber.ADS1299 = 30`
- `PP_SensorPartNumber.AFE4900 = 31`
- `PP_SensorPartNumber.MCP3008 = 32`
- `PP_SensorPartNumber.MCP3208 = 33`
- `PP_SensorPartNumber.ADS1220 = 34`
- `PP_SensorPartNumber.ADS131M08 = 35`
- `PP_SensorPartNumber.DAC8552 = 36`
- `PP_SensorPartNumber.MCP4922 = 37`
- `PP_SensorPartNumber.MAX30102 = 38`
- `PP_SensorPartNumber.MAX30105 = 39`
- `PP_SensorPartNumber.MAX86150 = 40`
- `PP_SensorPartNumber.AFE4404 = 41`
- `PP_SensorPartNumber.TMP117 = 42`
- `PP_SensorPartNumber.MCP9808 = 43`
- `PP_SensorPartNumber.SHT40 = 44`
- `PP_SensorPartNumber.SHT31 = 45`
- `PP_SensorPartNumber.HDC1080 = 46`
- `PP_SensorPartNumber.SI7021 = 47`
- `PP_SensorPartNumber.ADS1115 = 48`
- `PP_SensorPartNumber.NAU7802 = 49`
- `PP_SensorPartNumber.MCP4725 = 50`
- `PP_SensorPartNumber.MCP4728 = 51`
- `PP_SensorPartNumber.AD5693R = 52`

8.1.18 PP_ThresholdType

[PPL_API_1790]:

Name	PP_ThresholdType
Kind	enumeration
Elements	- <code>PP_ThresholdType.PP_THRESH_HARD = 0</code> - <code>PP_ThresholdType.PP_THRESH_SOFT = 1</code> - <code>PP_ThresholdType.PP_THRESH_GARROTE = 2</code>

Name	PP_ThresholdType
Description	How a denoising block treats coefficients below the threshold: zero them, shrink them all by the threshold, or shrink small ones hardest.
Available via	papaya (Python)

8.1.19 PP_Tune

[PPL_API_1795]:

Name	PP_Tune
Kind	enumeration
Elements	- PP_Tune.PP_LiveTune = 0 - PP_Tune.PP_NoLiveTune = 1
Description	Whether a block's parameters may be changed while the design runs.
Available via	papaya (Python)

8.1.20 PP_WaveForm

[PPL_API_1800]:

Name	PP_WaveForm
Kind	enumeration
Elements	- PP_WaveForm.Sine = 11 - PP_WaveForm.Tri = 12 - PP_WaveForm.Sqr = 13 - PP_WaveForm.Saw = 14 - PP_WaveForm.NoWaveForm = 65535 (unset)
Description	Choices accepted where a PP_WaveForm is expected.
Available via	papaya (Python)

8.1.21 PP_WaveMode

[PPL_API_1805]:

Name	PP_WaveMode
Kind	enumeration

Name	PP_WaveMode
Elements	- PP_WaveMode.Perpetual = 0 - PP_WaveMode.RepeatN = 1 - PP_WaveMode.OneShot = 2 - PP_WaveMode.TrigEdge = 3 - PP_WaveMode.TrigGate = 4 - PP_WaveMode.NoWaveMode = 255 (unset)
Description	How many times a generated waveform runs, and what starts it.
Available via	papaya (Python)

8.1.22 PP_WaveMultisinePhase

[PPL_API_1810]:

Name	PP_WaveMultisinePhase
Kind	enumeration
Elements	- PP_WaveMultisinePhase.Schroeder = 0 - PP_WaveMultisinePhase.Random = 1 - PP_WaveMultisinePhase.AllZero = 2 - PP_WaveMultisinePhase.NoWaveMultisinePhase = 255 (unset)
Description	How the phases of a multisine are chosen. Schroeder keeps the peak-to-average ratio low, which is what you want for excitation.
Available via	papaya (Python)

8.1.23 PP_WaveShape

[PPL_API_1815]:

Name	PP_WaveShape
Kind	enumeration
Elements	The 19 members are listed below the table.
Description	The shape a signal source generates.
Available via	papaya (Python)

- PP_WaveShape.Sine = 0
- PP_WaveShape.Square = 1
- PP_WaveShape.Triangle = 2
- PP_WaveShape.Sawtooth = 3
- PP_WaveShape.Pulse = 4

- `PP_WaveShape.Sinc = 5`
- `PP_WaveShape.Gauspuls = 6`
- `PP_WaveShape.DC = 7`
- `PP_WaveShape.Arbitrary = 8`
- `PP_WaveShape.LinearChirp = 9`
- `PP_WaveShape.LogChirp = 10`
- `PP_WaveShape.Multisine = 11`
- `PP_WaveShape.PRBS = 12`
- `PP_WaveShape.Step = 13`
- `PP_WaveShape.Ramp = 14`
- `PP_WaveShape.Impulse = 15`
- `PP_WaveShape.PulseTrain = 16`
- `PP_WaveShape.Dataset = 17`
- `PP_WaveShape.NoWaveShape = 65535 (unset)`

8.1.24 PP_WaveTrigPol

[PPL_API_1820]:

Name	PP_WaveTrigPol
Kind	enumeration
Elements	- <code>PP_WaveTrigPol.Rising = 0</code> - <code>PP_WaveTrigPol.Falling = 1</code> - <code>PP_WaveTrigPol.GateHigh = 2</code> - <code>PP_WaveTrigPol.GateLow = 3</code> - <code>PP_WaveTrigPol.NoWaveTrigPol = 255 (unset)</code>
Description	Which edge or level starts a triggered waveform.
Available via	papaya (Python)

8.1.25 PP_WaveletType

[PPL_API_1825]:

Name	PP_WaveletType
Kind	enumeration
Elements	The 10 members are listed below the table.
Description	Which wavelet the transform blocks use. The number in a name is the number of vanishing moments, so a higher one is smoother and has a longer support.
Available via	papaya (Python)

- `PP_WaveletType.HAAR = 0`

- `PP_WaveletType.DB4 = 1`
- `PP_WaveletType.DB6 = 2`
- `PP_WaveletType.DB8 = 3`
- `PP_WaveletType.DB10 = 4`
- `PP_WaveletType.SYM4 = 5`
- `PP_WaveletType.SYM8 = 6`
- `PP_WaveletType.COIF2 = 7`
- `PP_WaveletType.COIF4 = 8`
- `PP_WaveletType.NoWaveletType = 255 (unset)`

The wire-level enumerations (`PP_FuncId`, `PP_FuncParamId`, `PP_SubFuncId`) carry protocol values rather than caller choices. They are specified in the *PAPAYA DLP Programmer Manual*, which owns the wire contract.

8.2 Configuration structures

Blocks that need more than a handful of numbers take a structure. Every field is a Python float or sequence unless the table says otherwise, and a field left at its default is sent as that default rather than omitted.

8.2.1 PP_ACIM_EKF

[PPL_API_1830]:

Name	PP_ACIM_EKF
Kind	structure
Elements	The 10 fields are listed below the table.
Description	Sensorless AC induction motor EKF config.
Available via	papaya (Python)

- `Rs: float`
- `Rr: float`
- `Ls: float`
- `Lr: float`
- `Lm: float`
- `P_pairs: float`
- `Q_data: Sequence[float]`
- `R_data: Sequence[float]`
- `x_init: Sequence[float]`
- `P_init: float`

8.2.2 PP_ADRCCConfig

[PPL_API_1835]:

Name	PP_ADRCCConfig
Kind	structure
Elements	- b0: float - omega_o: float - omega_c: float - order: int
Description	Active Disturbance Rejection Control config.
Available via	papaya (Python)

8.2.3 PP_AHRS_EKF

[PPL_API_1840]:

Name	PP_AHRS_EKF
Kind	structure
Elements	- mag_ref: Sequence[float] - Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float
Description	AHRS quaternion estimator config.
Available via	papaya (Python)

8.2.4 PP_ActuatorSpec

[PPL_API_1845]:

Name	PP_ActuatorSpec
Kind	structure
Elements	- ConvConst1: float - LowerLimit: float - UpperLimit: float - HwId: PP_HwId
Description	Configuration accepted by the blocks that take a PP_ActuatorSpec.
Available via	papaya (Python)

8.2.5 PP_AdaptiveNotchConfig

[PPL_API_1850]:

Name	PP_AdaptiveNotchConfig
Kind	structure
Elements	- init_freq: float - Q_factor: float - mu: float
Description	Adaptive notch filter config.
Available via	papaya (Python)

8.2.6 PP_BaroAlt_EKF**[PPL_API_1855]:**

Name	PP_BaroAlt_EKF
Kind	structure
Elements	- Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float
Description	Barometric altitude EKF config.
Available via	papaya (Python)

8.2.7 PP_BatterySOC_EKF**[PPL_API_1860]:**

Name	PP_BatterySOC_EKF
Kind	structure
Elements	The 9 fields are listed below the table.
Description	Battery SOC estimator config (1-RC Thevenin).
Available via	papaya (Python)

- R0: float
- R1: float
- C1: float
- Q_cap: float
- ocv_coef: Sequence[float]
- Q_data: Sequence[float]
- R_data: Sequence[float]
- x_init: Sequence[float]
- P_init: float

8.2.8 PP_BatterySOH_EKF

[PPL_API_1865]:

Name	PP_BatterySOH_EKF
Kind	structure
Elements	The 7 fields are listed below the table.
Description	Battery SOH EKF config.
Available via	papaya (Python)

- R1: float
- C1: float
- ocv_coef: Sequence[float]
- Q_data: Sequence[float]
- R_data: Sequence[float]
- x_init: Sequence[float]
- P_init: float

8.2.9 PP_DOBConfig

[PPL_API_1870]:

Name	PP_DOBConfig
Kind	structure
Elements	- Gn_num: Sequence[float] - Gn_den: Sequence[float] - Qf_num: Sequence[float] - Qf_den: Sequence[float] - plant_ord: int - qf_ord: int
Description	Disturbance observer config.
Available via	papaya (Python)

8.2.10 PP_ESCConfig

[PPL_API_1875]:

Name	PP_ESCConfig
Kind	structure
Elements	- a: float - omega: float - k: float - hpf_wc: float - lpf_wc: float
Description	Extremum seeking control config.
Available via	papaya (Python)

8.2.11 PP_ESC_PID_Config

[PPL_API_1880]:

Name	PP_ESC_PID_Config
Kind	structure
Elements	The 11 fields are listed below the table.
Description	Extremum Seeking PID Optimizer config.
Available via	papaya (Python)

- Kp_init: float
- Ki_init: float
- Kd_init: float
- a: float
- omega_p: float
- omega_i: float
- omega_d: float
- esc_gain: float
- lpf_wc: float
- Tau: float
- iae_window: float

8.2.12 PP_FuzzyPIDConfig

[PPL_API_1885]:

Name	PP_FuzzyPIDConfig
Kind	structure
Elements	The 8 fields are listed below the table.
Description	Fuzzy logic PID config.
Available via	papaya (Python)

- Kp_base: float
- Ki_base: float
- Kd_base: float
- e_range: float
- de_range: float
- Tau: float
- out_min: float
- out_max: float

8.2.13 PP_GPSINS_EKF

[PPL_API_1890]:

Name	PP_GPSINS_EKF
Kind	structure
Elements	- Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float
Description	GPS/INS fusion EKF config.
Available via	papaya (Python)

8.2.14 PP_GainSchedulePIDConfig

[PPL_API_1895]:

Name	PP_GainSchedulePIDConfig
Kind	structure
Elements	The 8 fields are listed below the table.
Description	Gain-scheduled PID config.
Available via	papaya (Python)

- schedule_points: Sequence[float]
- Kp_table: Sequence[float]
- Ki_table: Sequence[float]
- Kd_table: Sequence[float]
- Tau: float
- out_min: float
- out_max: float
- num_points: int

8.2.15 PP_GainSchedulerConfig

[PPL_API_1900]:

Name	PP_GainSchedulerConfig
Kind	structure
Elements	- breakpoints: Sequence[float] - values: Sequence[float] - num_points: int - num_outputs: int
Description	Generic gain scheduler config.
Available via	papaya (Python)

8.2.16 PP_GridSync_EKF

[PPL_API_1905]:

Name	PP_GridSync_EKF
Kind	structure
Elements	- nom_freq: float - nom_amplitude: float - Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float
Description	Grid frequency and phase tracker EKF config.
Available via	papaya (Python)

8.2.17 PP_ILCConfig

[PPL_API_1910]:

Name	PP_ILCConfig
Kind	structure
Elements	- trial_length: int - L_gain: float - Q_gain: float
Description	Iterative Learning Control config.
Available via	papaya (Python)

8.2.18 PP_LMSConfig

[PPL_API_1915]:

Name	PP_LMSConfig
Kind	structure
Elements	- mu: float - N: int - normalize: int - delta: float
Description	LMS / NLMS adaptive filter config.
Available via	papaya (Python)

8.2.19 PP_LoadTorque_EKF

[PPL_API_1920]:

Name	PP_LoadTorque_EKF
Kind	structure
Elements	- J: float - B: float - Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float
Description	Load torque estimator config.
Available via	papaya (Python)

8.2.20 PP_MRACConfig

[PPL_API_1925]:

Name	PP_MRACConfig
Kind	structure
Elements	- Am: float - Bm: float - gamma_x: float - gamma_r: float
Description	Model Reference Adaptive Control config.
Available via	papaya (Python)

8.2.21 PP_MRACLyapunovConfig

[PPL_API_1930]:

Name	PP_MRACLyapunovConfig
Kind	structure
Elements	- Am: float - Bm: float - gamma_x: float - gamma_r: float - sigma: float - theta_max: float
Description	MRAC with Lyapunov stability guarantee config.
Available via	papaya (Python)

8.2.22 PP_MRAC_PID_Config

[PPL_API_1935]:

Name	PP_MRAC_PID_Config
Kind	structure
Elements	- Am: float - Bm: float - gamma_p: float - gamma_i: float - gamma_d: float - Tau: float

Name	PP_MRAC_PID_Config
Description	Model Reference Adaptive PID config.
Available via	papaya (Python)

8.2.23 PP_PMSM_EKF

[PPL_API_1940]:

Name	PP_PMSM_EKF
Kind	structure
Elements	The 7 fields are listed below the table.
Description	Sensorless PMSM observer config.
Available via	papaya (Python)

- Rs: float
- Ls: float
- lambda_pm: float
- Q_data: Sequence[float]
- R_data: Sequence[float]
- x_init: Sequence[float]
- P_init: float

8.2.24 PP_PerfMonPID_Config

[PPL_API_1945]:

Name	PP_PerfMonPID_Config
Kind	structure
Elements	The 8 fields are listed below the table.
Description	Performance-Monitoring Adaptive PID config.
Available via	papaya (Python)

- Kp_init: float
- Ki_init: float
- Kd_init: float
- Tau: float
- iae_threshold: float
- out_min: float
- out_max: float
- eval_window: float

8.2.25 PP_RLSPParam

[PPL_API_1950]:

Name	PP_RLSPParam
Kind	structure
Elements	The 7 fields are listed below the table.
Description	Configuration accepted by the blocks that take a PP_RLSPParam.
Available via	papaya (Python)

- num: Sequence[float]
- lenNum: int
- denom: Sequence[float]
- lenDenom: int
- InitVariance_f32: float
- ModelOrder_u8: int
- UpdateDivider_u16: int (default 1)

8.2.26 PP_RRLSPParam

[PPL_API_1955]:

Name	PP_RRLSPParam
Kind	structure
Elements	The 8 fields are listed below the table.
Description	Configuration accepted by the blocks that take a PP_RRLSPParam.
Available via	papaya (Python)

- num: Sequence[float]
- lenNum: int
- denom: Sequence[float]
- lenDenom: int
- InitVariance_f32: float
- ModelOrder_u8: int
- sensParam_f32: float
- UpdateDivider_u16: int (default 1)

8.2.27 PP_RelayAutoTuneConfig

[PPL_API_1960]:

Name	PP_RelayAutoTuneConfig
Kind	structure
Elements	- relay_amplitude: float - hysteresis: float - setpoint: float - tuning_rule: int - min_cycles: int
Description	Relay feedback PID auto-tuner config.
Available via	papaya (Python)

8.2.28 PP_RepCtrlConfig

[PPL_API_1965]:

Name	PP_RepCtrlConfig
Kind	structure
Elements	- period_samples: int - Q_gain: float - Kr: float
Description	Repetitive controller config.
Available via	papaya (Python)

8.2.29 PP_RobotLoc_EKF

[PPL_API_1970]:

Name	PP_RobotLoc_EKF
Kind	structure
Elements	The 8 fields are listed below the table.
Description	2D robot localization EKF config.
Available via	papaya (Python)

- wheel_radius: float
- wheel_base: float
- landmark_x: float
- landmark_y: float
- Q_data: Sequence[float]
- R_data: Sequence[float]
- x_init: Sequence[float]
- P_init: float

8.2.30 PP_SMO

[PPL_API_1975]:

Name	PP_SMO
Kind	structure
Elements	The 10 fields are listed below the table.
Description	Configuration for the sliding mode observer node (SISO / MIMO).
Available via	papaya (Python)

- n: int
- m: int
- p: int
- A_data: Sequence[float]
- B_data: Sequence[float]
- C_data: Sequence[float]
- L_data: Sequence[float]
- K_data: Sequence[float]
- x_init: Sequence[float]
- epsilon: float

8.2.31 PP_STRConfig

[PPL_API_1980]:

Name	PP_STRConfig
Kind	structure
Elements	- na: int - nb: int - lambda_: float - P_init: float - desired_poles: Sequence[float]
Description	Self-Tuning Regulator config.
Available via	papaya (Python)

8.2.32 PP_STR_PID_Config

[PPL_API_1985]:

Name	PP_STR_PID_Config
Kind	structure
Elements	- lambda_: float - P_init: float - Tau: float - desired_cl_bw: float - UpdateDivider_u16: int (default 1)

Name	PP_STR_PID_Config
Description	Self-Tuning PID config.
Available via	papaya (Python)

8.2.33 PP_SensorSpec

[PPL_API_1990]:

Name	PP_SensorSpec
Kind	structure
Elements	- ConvConst1: float - ConvConst2: float - HwId: PP_HwId
Description	Configuration accepted by the blocks that take a PP_SensorSpec.
Available via	papaya (Python)

8.2.34 PP_SinTrack_EKF

[PPL_API_1995]:

Name	PP_SinTrack_EKF
Kind	structure
Elements	- Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float
Description	Sinusoidal signal tracker EKF config.
Available via	papaya (Python)

8.2.35 PP_StepAutoTuneConfig

[PPL_API_2000]:

Name	PP_StepAutoTuneConfig
Kind	structure
Elements	- step_size: float - settle_pct: float - tuning_rule: int - id_method: int
Description	Step response PID auto-tuner config.
Available via	papaya (Python)

8.2.36 PP_Thermal_EKF

[PPL_API_2005]:

Name	PP_Thermal_EKF
Kind	structure
Elements	The 9 fields are listed below the table.
Description	Thermal model EKF config (2-node lumped RC).
Available via	papaya (Python)

- R_th: float
- C_core: float
- C_surf: float
- R_conv: float
- T_amb: float
- Q_data: Sequence[float]
- R_data: Sequence[float]
- x_init: Sequence[float]
- P_init: float

8.2.37 PP_Tilt_EKF

[PPL_API_2010]:

Name	PP_Tilt_EKF
Kind	structure
Elements	- Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float
Description	Single-axis tilt estimator config.
Available via	papaya (Python)

8.2.38 PP_WMR_EKF

[PPL_API_2015]:

Name	PP_WMR_EKF
Kind	structure
Elements	- wheel_base: float - mag_ref: Sequence[float] - Q_data: Sequence[float] - R_data: Sequence[float] - x_init: Sequence[float] - P_init: float

Name	PP_WMR_EKF
Description	Wheeled Mobile Robot EKF config.
Available via	papaya (Python)

8.2.39 PP_adaptiveKF

[PPL_API_2020]:

Name	PP_adaptiveKF
Kind	structure
Elements	The 11 fields are listed below the table.
Description	Configuration accepted by the blocks that take a PP_adaptiveKF.
Available via	papaya (Python)

- dim_x: int
- dim_z: int
- dim_u: int
- x_init: Sequence[float]
- F_data: Sequence[float]
- H_data: Sequence[float]
- Q_data: Sequence[float]
- B_data: Optional[Sequence[float]]
- R_init: float
- P_init: float
- alpha: float

8.2.40 PP_anInCfg

[PPL_API_2025]:

Name	PP_anInCfg
Kind	structure
Elements	- outScale: float - offset: float - id: PP_HwId
Description	Configuration accepted by the blocks that take a PP_anInCfg.
Available via	papaya (Python)

8.2.41 PP_anOutCfg

[PPL_API_2030]:

Name	PP_anOutCfg
Kind	structure
Elements	- lowerLimit: float - upperLimit: float - id: PP_HwId
Description	One row of a multi-channel input-driven analog output block.
Available via	papaya (Python)

8.2.42 PP_compFilter

[PPL_API_2035]:

Name	PP_compFilter
Kind	structure
Elements	- alpha: float
Description	Configuration for the complementary filter node.
Available via	papaya (Python)

8.2.43 PP_encoderBlockCfg

[PPL_API_2040]:

Name	PP_encoderBlockCfg
Kind	structure
Elements	- id: PP_HwId - config: PP_encoderCfg - outputCount: int - outputs: Sequence['PP_EncoderOutput']
Description	Full encoder block descriptor: identifier + config + output selection.
Available via	papaya (Python)

8.2.44 PP_encoderCfg

[PPL_API_2045]:

Name	PP_encoderCfg
Kind	structure
Elements	- vel_conv: float - pos_conv: float - vel_tau: float (default 0.01)
Description	Configuration parameters for a standalone quadrature encoder block.
Available via	papaya (Python)

8.2.45 PP_linearKF**[PPL_API_2050]:**

Name	PP_linearKF
Kind	structure
Elements	The 10 fields are listed below the table.
Description	Configuration accepted by the blocks that take a PP_linearKF.
Available via	papaya (Python)

- dim_x: int
- dim_z: int
- dim_u: int
- x_init: Sequence[float]
- F_data: Sequence[float]
- H_data: Sequence[float]
- Q_data: Sequence[float]
- B_data: Optional[Sequence[float]]
- R_init: float
- P_init: float

8.2.46 PP_motorBlockCfg**[PPL_API_2055]:**

Name	PP_motorBlockCfg
Kind	structure
Elements	- id: PP_HwId - config: PP_motorCfg - outputCount: int - outputs: Sequence[PP_MotorOutput]

Name	PP_motorBlockCfg
Description	Full motor block descriptor: identifier + config + output selection.
Available via	papaya (Python)

8.2.47 PP_motorCfg

[PPL_API_2060]:

Name	PP_motorCfg
Kind	structure
Elements	The 8 fields are listed below the table.
Description	Configuration parameters for a brushed-DC motor channel.
Available via	papaya (Python)

- pwm_hz: float
- cur_min_mA: float
- cur_max_mA: float
- vel_conv: float
- pos_conv: float
- vel_tau: float
- dutyLowerLimit: float
- dutyUpperLimit: float

8.2.48 PP_pwmInCfg

[PPL_API_2065]:

Name	PP_pwmInCfg
Kind	structure
Elements	- outScale: float - id: PP_HwId
Description	One row of a multi-channel :class:papaya.blocks.pwmIn block.
Available via	papaya (Python)

8.2.49 PP_pwmOutCfg

[PPL_API_2070]:

Name	PP_pwmOutCfg
Kind	structure
Elements	- frequency: float - dutyLowerLimit: float - dutyUpperLimit: float - id: PP_HwId
Description	One row of a multi-channel :class:papaya.blocks.pwmOut block.
Available via	papaya (Python)

8.2.50 PP_stateSpace**[PPL_API_2075]:**

Name	PP_stateSpace
Kind	structure
Elements	The 8 fields are listed below the table.
Description	Configuration for the state-space system node (SISO / MIMO).
Available via	papaya (Python)

- n: int
- m: int
- p: int
- A_data: Sequence[float]
- B_data: Sequence[float]
- C_data: Sequence[float]
- D_data: Sequence[float]
- x_init: Sequence[float]

Chapter 9

Linking blocks

Links are made with two operators.

Operator	Meaning
<code>a > b</code>	Route a's scalar output into b's input
<code>a >> v</code>	Route a frame-carrying output into a vector probe
<code>a.out(i) > b.in_(j)</code>	Address a specific port by index (Python)
<code>a.out(i) > b.in(j)</code>	The same in C++

```
src > filt           # single output to single input
src.out(0) > mux.in_(2) # explicit ports
fft >> spectrum     # a frame to a vector probe
```

Warning

An output drives exactly one input. The second link is refused, not silently ignored. Fan out with expander:

```
fan = expander(3)
src > fan
fan.out(0) > a
fan.out(1) > b
fan.out(2) > c
```

Chapter 10

Configuration structures

Blocks with many settings take a configuration struct rather than a long argument list. The pattern is identical in both languages: fill the struct, pass it to the constructor.

```
cfg = PP_Thermal_EKF(R_th=0.5, C_core=100.0, C_surf=50.0,  
                    R_conv=5.0, T_amb=25.0,  
                    Q_data=[0.01, 0.0, 0.0, 0.01], R_data=[0.25],  
                    x_init=[25.0, 25.0], P_init=1.0)  
ekf = thermalEKF(cfg)
```

Note

Array fields such as `Q_data` are **row-major** and must match the state dimension. A short array is not padded.